OneSumX Regulatory Engine 4.9.10
ReportingHub REST API

Wolters Kluwer

Copyright statement

Copyright © 2025 Wolters Kluwer Financial Services Belgium NV (“Wolters Kluwer Financial Services”). All rights reserved.

All other registered or unregistered trademarks and service marks are property of their respective companies and should be treated as such.

No part of this publication may be reproduced, transcribed, transmitted, stored in a retrieval system,computer or otherwise, in any form or by any means, magnetic, mechanical, electronic, optical, manual or otherwise, and may not be translated into any language without the express written permission of Wolters Kluwer Financial Services.

Limit of Liability - Disclaimer of Warranty

Wolters Kluwer Financial Services makes no representations or warranties of any kind with respect to this manual and the contents hereof and specifically disclaim any warranties, either expressed or implied, or merchantability or fitness for any particular purpose. In no event shall Wolters Kluwer Financial Services or its subsidiaries be held liable for errors contained herein or direct, indirect, special, incidental, or consequential damages in connection with or arising from the use of the product, the accompanying manual, or any related materials. Further, Wolters Kluwer Financial Services reserves the right to change both this publication and the software programs to which it relates and to make changes from time to time to the content hereof with no obligation to notify any person or organization of such revision or changes.

All Wolters Kluwer Financial Services publications and computer programs contain proprietary confidential information of Wolters Kluwer Financial Services and their possession and use are subject to restrictions set forth in the license agreement entered into between Wolters Kluwer Financial Services and its licensees. No title or ownership of Wolters Kluwer Financial Services software is transferred, and any use of the product and its related materials beyond the terms on this license, without the written authorization of Wolters Kluwer Financial Services, is prohibited.

Export Restrictions

The recipient agrees to comply in all respects with any governmental laws, orders, other restrictions (“Export Restrictions”) on the export or re-export of the software or related documentation imposed by any applicable public authority (including the government of the United States, the governments of the member states of the European Union and other countries in which the recipient is located). The recipient shall not commit any act or omission that will result in a breach of any such export restrictions.

Statement of Confidentiality

The recipient of this document agrees to keep this document confidential and not to use its contents in any form or by any means without prior written permission from Wolters Kluwer Financial Services.

If such permission is granted, it will be subject to the condition that the recipient ensures that any third party recipient of this document, or of information contained therein, is held responsible to Wolters Kluwer Financial Services for the confidentiality of that document or information.

Gender

Throughout this document the user/reader is referred to as he/him/his. We mean this to include she/her/hers equally.

Wolters Kluwer

Table of contents

1	**Introduction**	**7**
1.1	Audience	7
2	**General**	**8**
2.1	REST interfaces	8
2.2	Status codes	8
3	**Admin center API**	**10**
3.1	Package Manager API	10
3.1.1	Upload Reporting module	10
3.1.1.1	Request	10
3.1.1.2	Response	10
3.1.2	Reporting module ID retrieval	10
3.1.2.1	Request	11
3.1.2.2	Response	11
3.2	Job API	11
3.2.1	Get job status	11
3.2.1.1	Request	11
3.2.1.2	Response	11
3.3	Workspace API	12
3.3.1	Create Workspace	12
3.3.1.1	Request	12
3.3.1.2	Example - request: create workspace Oracle database	14
3.3.1.3	Example - request: create workspace MS SQL database	15
3.3.1.4	Example - request: create workspace with Shared GDM	15
3.3.1.5	Example - request: attach workspace	15
3.3.1.6	Example - request: create workspace with Integrated Security	16
3.3.1.7	Response	16
3.3.2	Workspace status	16
3.3.2.1	Request	17
3.3.2.2	Response	17
3.3.3	Upgrade workspace	17
3.3.3.1	Request	17
3.3.3.2	Example: upgrade GDM LDM workspace on Oracle	18
3.3.3.3	Example: upgrade LDM only workspace on Oracle	18
3.3.3.4	Example: upgrade workspace on MS SQL Server using login/password authentication	18
3.3.3.5	Example: upgrade workspace on MS SQL Server using integrated security	18
3.3.3.6	Response	18
3.3.4	Upgrade unmanaged workspaces sharing GDM	18
3.3.4.1	Request	19
3.3.4.2	Definition table of nested object in workspaces field	19
3.3.4.3	Example: upgrade unmanaged workspaces sharing GDM on Oracle	19
3.3.4.4	Example: upgrade unmanaged workspaces sharing GDM on MS SQL Server using login/password authentication	20
3.3.4.5	Response	20
3.3.5	Optimize workspace	20
3.3.5.1	Request	20
3.3.5.2	Example: optimize GDM LDM workspace on Oracle	21
3.3.5.3	Example: optimize LDM only workspace on Oracle	21
3.3.5.4	Example: optimize workspace on MS SQL Server using login/password authentication	21
3.3.5.5	Example: optimize workspace on MS SQL Server using integrated security	21
3.3.5.6	Response	22
3.4	Resource Group API	22
3.4.1	Upgrade Resource Group	22
3.4.1.1	Request	22
4	**Reporting hub API**	**24**
4.1	Submission template API	24
4.1.1	Request	24
4.1.1.1	Definition table of nested object in selectedReports field	24

4.1.2	Response	25
4.2	Data set API	27
4.2.1	Create data set	27
4.2.1.1	Request	27
4.2.1.2	Response	27
4.2.2	Get data sets	28
4.2.2.1	Request	28
4.2.2.2	Response	28
4.2.3	Discard data set	29
4.2.3.1	Request	30
4.2.3.2	Response	30
4.2.4	Get data set ID	30
4.2.4.1	Request	31
4.2.4.2	Response	31
4.2.5	Trigger data set input data validation process	31
4.2.5.1	Request	31
4.2.5.2	Response	32
4.2.6	Retrieve data set input data validation status	32
4.2.6.1	Request	32
4.2.6.2	Response	32
4.2.7	Get process parameter set ID	33
4.2.7.1	Request	33
4.2.7.2	Response	33
4.2.8	Trigger data set enrichment process	33
4.2.8.1	Request	33
4.2.8.2	Response	34
4.2.9	Get list of result sets	34
4.2.9.1	Request	34
4.2.9.2	Response	35
4.3	Submission API	36
4.3.1	Create submission	36
4.3.1.1	Request	36
4.3.1.2	Response	37
4.3.2	Get submission	37
4.3.2.1	Request	37
4.3.2.2	Response	38
4.3.3	Discard submission	40
4.3.3.1	Request	40
4.3.3.2	Response	40
4.3.4	Assign result sets to submission	40
4.3.4.1	Request	40
4.3.4.2	Response	41
4.3.5	Count assigned result sets	41
4.3.5.1	Request	41
4.3.5.2	Response	41
4.3.6	Get post process ids by report set ids	41
4.3.6.1	Request	42
4.3.6.2	Response	42
4.3.7	Postprocess	42
4.3.7.1	Request	42
4.3.7.2	Response	42
4.3.8	Trigger submission allocation process	43
4.3.8.1	Request	43
4.3.8.2	Response	43
4.3.9	Trigger submission validation process	43
4.3.9.1	Request	43
4.3.9.2	Response	44
4.3.10	Trigger delivery process	44
4.3.10.1	Request	44
4.3.10.2	Response	44
4.3.11	Get transmission artifacts	44

4.3.11.1	Request	45
4.3.11.2	Response	45
4.3.12	Get transmission file	45
4.3.12.1	Request	45
4.3.12.2	Response	46
4.3.13	Add submission references to submission	46
4.3.13.1	Request	46
4.3.13.2	Response	46
4.3.14	Get submission references	47
4.3.14.1	Request	47
4.3.14.2	Response	47
4.3.15	Get available submissions	47
4.3.15.1	Request	47
4.3.15.2	Response	48
4.3.16	Remove submission references from submission	49
4.3.16.1	Request	49
4.3.16.2	Response	49
4.4	Job API	49
4.4.1	Get job status	49
4.4.1.1	Request	49
4.4.1.2	Response	49
4.4.2	Get job status (recommeded)	50
4.4.2.1	Request	50
4.4.2.2	Response	50
4.5	Bridge structure API	50
4.5.1	Prepare data structure to launch bridge	50
4.5.1.1	Request	50
4.5.1.2	Response	51
4.5.2	Prepare data structure to launch bridge with multiple data units	51
4.5.2.1	Request for one or more data units	51
4.5.2.2	Response	51
5	**Admin Center PowerShell script samples**	**53**
5.1	Utility Scripts	53
5.1.1	Constants	53
5.1.2	Lookup workspace status	53
5.2	Reporting module uploading script	55
5.3	Workspace creation script	58
5.4	Workspace upgrading script	62
5.5	Multiple unmanaged workspaces sharing the same GDM upgrading script	64
5.6	Workspace optimization script	67
5.7	Resource Group update script	70
6	**Reporting Hub PowerShell script samples**	**74**
6.1	Main script	74
6.2	Utility scripts	78
6.2.1	ProcessParameters	78
6.2.2	Constants	78
6.3	Create submission template script	80
6.4	Create data set script	82
6.5	Discard data set	85
6.6	Lookup data set ID	88
6.7	Validate input data	90
6.8	Enrich input data	92
6.9	Lookup created result sets	95
6.10	Add submission	96
6.11	Lookup submission ID	98
6.12	Add data set to submission	100
6.13	Discard submission	102
6.14	Submission references	104
6.14.1	Add submission references to submission	104
6.14.2	Remove submission references from submission	105
6.15	Post process	106

6.15.1	Lookup after post process	106
6.15.2	Call the post processes	108
6.16	Allocate reports	110
6.17	Validate reports	112
6.18	Create transmission file	114
6.19	Lookup and download the transmission files	116
6.20	Prepare bridge structure	118
6.21	Skip SSL validation	119

1 Introduction

1.1 Audience

This guide is intended primarily for IT administrators, developers, and devops.

2 General

2.1 REST interfaces

The user interface of the OneSumX Regulatory Engine solution interacts with the back end using a REST interface. This REST interface can be used to automate the OneSumX Regulatory Engine solution, by means of various tools that allow interaction with a REST interface. The following tools are suitable candidates for using the OneSumX Regulatory Engine REST interface:

- PowerShell
- cURL
- PostMan

All REST endpoints in the OneSumX Regulatory Engine solution use JSON payloads for both requests and responses.

All REST endpoints in the OneSumX Regulatory Engine solution are protected by the authentication mechanism of the solution. Following the login, the user's browser receives an access token from the solution in the form of a cookie, which is then used by the browser for all REST calls of the UI towards the back end. The validity of such a token is limited in time.

For automation purposes, the OneSumX Regulatory Engine solution provides the possibility to issue an API key (see OneSumX Regulatory Engine Administrator User Manual). The API key is required to be sent with each request as Wolters Kluwer Token cookie.

Warning:
Tokens created in previous versions of OneSumX Regulatory Engine (prior to 4.2.0) are not compatible with RE 4.2.x and must be regenerated.

Tip:
An example of API key usage can be found in topic Admin Center PowerShell script samples.

2.2 Status codes

All REST interfaces have a common approach regarding HTTP status codes. The following can be applied to any of the described REST interfaces:

Status code	Reason
200	**The request has succeeded. The information returned with the response is dependent on the method used in the request.**
201	**The request has been fulfilled and resulted in a new resource being created.**
204	**The server has fulfilled the request but does not need to return an entity-body.**

Status code	Reason
400	**The request could not be understood by the server due to malformed syntax. Check if all required parameters are present and have correct format.**
401	**The request requires user authentication. Make sure valid API key was used to perform request.**
403	**The server understood the request but is refusing to fulfill it.**
404	**Resource not found.**
500	**The server encountered an unexpected condition which prevented it from fulfilling the request. Details can be found in server logs.**

3 Admin center API

3.1 Package Manager API

3.1.1 Upload Reporting module

It is possible to upload a reporting module by provided it`s path to file system.

```
POST /admin-api/external/v1_0/package-manager/modules
```

This endpoint is intended to be used in combination with Job status retrieval and uploaded Reporting module ID retrieval.

3.1.1.1 Request

Name	Located in	Required	Description	Default	Schema
reportingModulePath	**Body**	**yes**	**Path to Reporting module file**	**-**	**String**

The Content-Type header parameter value should be multipart/form-data.

The format of the reportingModulePath parameter value should be as the following: file: *\\<computer_full_name>\C$\<path_to_reporting_module>*.

3.1.1.2 Response

Name	Description	Schema
status	**Uploading Job ID**	**Long**

```
{
    "status": "1"
}
```

3.1.2 Reporting module ID retrieval

This request gets a reporting module ID by parameters with short name and version of existed reporting module.

```
GET /admin-api/external/v1_0/package-manager/status
```

3.1.2.1 Request

Name	Located in	Required	Description	Default	Schema
shortName	**Query**	**yes**	**Short name of uploading reporting module**	**-**	**String**
version	**Query**	**yes**	**Version of uploading reporting module**	**-**	**String**

3.1.2.2 Response

Name	Description	Schema
ID	**Reporting module ID**	**Long**

```
1
```

3.2 Job API

3.2.1 Get job status

```
GET /admin-api/external/v1_0/jobs/${jobId}
```

3.2.1.1 Request

Name	Located in	Required	Description	Default	Schema
jobId	**Path**	**yes**	**ID of the uploading reporting module job**	**-**	**Long**

3.2.1.2 Response

Name	Description	Schema
status	**Status of the uploading job**	**String**

```
FINISHED
```

3.3 Workspace API

3.3.1 Create Workspace

```
POST /admin-api/external/v1_0/workspaces
```

3.3.1.1 Request

Name	Located in	Required	Description	Default	Schema
name	**body**	**yes**	**Unique name of the workspace**	**-**	**String**
description	**body**	**yes**	**Description**	**-**	**String**
workspaceType	**body**	**yes**	**Type of the workspace. Available values: GDM_LDM, LDM**	**-**	**String**
machineName	**body**	**yes**	**Database connection property. Host name**	**-**	**String**
instanceName	**body**	**no**	**Database connection property. MSSQL only.**	**-**	**String**
serviceName	**body**	**no**	**Database connection property. Oracle only.**	**-**	**String**
tnsServiceName	**body**	**no**	**Database connection property. Oracle only.**	**-**	**String**
port	**body**	**yes**	**Database connection property**	**-**	**String**
connectionUsername	**body**	**no**	**Database connection property**	**-**	**String**
connectionPassword	**body**	**no**	**Database connection property**	**-**	**String**
administratorUsername	**body**	**no**	**Database connection property**	**-**	**String**

Name	Located in	Required	Description	Default	Schema
administratorPassword	body	no	Database connection property	-	String
gdmSchemaPassword	body	no	Database connection property	-	String
gdmSchemaName	body	no	Database connection property	-	String
ldmSchemaPassword	body	no	Database connection property	-	String
ldmSchemaName	body	yes	Database connection property	-	String
odbcDriver	body	yes	Database connection property	-	String
vendor	body	yes	Database connection property. Available values: ORACLE, SQLSERVER	-	String
locationLink	body	no	Database connection property. For MSSQL Server location link must contain trustServerCertificate	-	String
sharedRODirectoryRoot	body	no	Attach workspace operation property.	-	String
newSharedRODirectoryRoot	body	no	Attach workspace operation property.	-	String
newDb	body	yes	Attach workspace operation property	-	Boolean
sharedWorkspaces	body	no	Workspace with shared GDM	-	String
gdmApproach	body	yes	Shared GDM property. Available values:	-	String

Name	Located in	Required	Description	Default	Schema
			NewGdm, SharedGdm		
shortName	**body**	**no**	**Reporting module property.** **Reporting module short name.**	**-**	**String**
version	**body**	**no**	**Reporting module property.** **Reporting module version.**	**-**	**String**
targetApplication	**body**	**no**	**Application where workspace will be used**	**-**	**String**
reportingModulePath	**body**	**yes**	**Absolute path to Reporting module file**	**-**	**String**
trusetServerCertificate	**body**	**no**	**Mandatory ONLY for MSSQL server**	**-**	**Boolean**

3.3.1.2 Example - request: create workspace Oracle database

```
{
    "name": "Workspace Name",
    "description": "Workspace Description",
    "workspaceType": "GDM_LDM",
    "targetApplication": "Reporting Hub",
    "newDb": true,
    "shortName": "reporting_module_shortname",
    "version": "reporting_module_version",
    "vendor": "ORACLE",
    "machineName": "localhost",
    "serviceName": "ORACLE",
    "tnsServiceName": "ORACLE",
    "port": "1521",
    "odbcDriver": "Oracle in OraClient19Home1_32bit",
    "locationLink": "jdbc:oracle:thin:@//localhost:1521/ORACLE",
    "administratorUsername": "database_administrator_username",
    "administratorPassword": "database_administrator_password",
    "gdmSchemaPassword": "gdm_password",
    "gdmSchemaName": "GDM_SCHEMA_NAME",
    "ldmSchemaPassword": "ldm_password",
    "ldmSchemaName": "LDM_SCHEMA_NAME",
    "gdmApproach": "NewGdm",
    "connectionUsername": "",
    "connectionPassword": ""
}
```

3.3.1.3 Example - request: create workspace MS SQL database

```
{
    "name": "Workspace Name",
    "description": "Workspace Description",
    "workspaceType": "GDM_LDM",
    "targetApplication": "Reporting Hub",
    "newDb": true,
    "shortName": "reporting_module_shortname",
    "version": "reporting_module_version",
    "vendor": "SQLSERVER",
    "machineName": "localhost",
    "instanceName": "SQL2022_CI",
    "port": "59425",
    "odbcDriver": "ODBC Driver 17 for SQL Server",,
    "locationLink": "jdbc:sqlserver://localhost:59425;instanceName=SQL2022_CI;trustS
erverCertificate=true;",
    "administratorUsername": "database_administrator_username",
    "administratorPassword": "database_administrator_password",
    "gdmSchemaPassword": "gdm_password",
    "gdmSchemaName": "GDM_SCHEMA_NAME",
    "ldmSchemaPassword": "ldm_password",
    "ldmSchemaName": "LDM_SCHEMA_NAME",
    "gdmApproach": "NewGdm",
    "trustServerCertificate": true
}
```

3.3.1.4 Example - request: create workspace with Shared GDM

```
{
    "name": "Workspace Name",
    "description": "Workspace Description",
    "workspaceType": "GDM_LDM",
    "targetApplication": "Reporting Hub",
    "newDb": true,
    "shortName": "reporting_module_shortname",
    "version": "reporting_module_version",
    "vendor": "ORACLE",
    "machineName": "localhost",
    "serviceName": "ORACLE",
    "tnsServiceName": "ORACLE",
    "port": "1521",
    "odbcDriver": "Oracle in OraClient19Home1_32bit",
    "locationLink": "jdbc:oracle:thin:@//localhost:1521/ORACLE",
    "administratorUsername": "database_administrator_username",
    "administratorPassword": "database_administrator_password",
    "sharedWorkspaces": "workspace_name/reporting_module_name
reporting_module_version",
    "gdmSchemaPassword": "gdm_password",
    "gdmSchemaName": "GDM_SCHEMA_NAME",
    "ldmSchemaPassword": "ldm_password",
    "ldmSchemaName": "LDM_SCHEMA_NAME",
    "gdmApproach": "SharedGdm",
    "connectionUsername": "",
    "connectionPassword": ""
}
```

3.3.1.5 Example - request: attach workspace

```
{
```

```
    "name": "Workspace Name",
    "description": "Workspace Description",
    "workspaceType": "GDM_LDM",
    "targetApplication": "Reporting Hub",
    "newDb": false,
    "vendor": "ORACLE",
    "machineName": "localhost",
    "serviceName": "ORACLE",
    "tnsServiceName": "ORACLE",
    "port": "1521",
    "odbcDriver": "Oracle in OraClient19Home1_32bit",
    "gdmSchemaPassword": "gdm_password",
    "gdmSchemaName": "existing_gdm_schema_name",
    "ldmSchemaPassword": "ldm_password",
    "ldmSchemaName": "existing_ldm_schema_name",
    "sharedRODirectoryRoot": "old_artifact_ldm_path",
    "newSharedRODirectoryRoot": "new_artifact_ldm_path",
    "gdmApproach": "NewGdm",
    "connectionUsername": "",
    "connectionPassword": ""
}
```

3.3.1.6 Example - request: create workspace with Integrated Security

```
{
    "name": "Workspace Name",
    "description": "Workspace Description",
    "workspaceType": "GDM_LDM",
    "targetApplication": "Reporting Hub",
    "newDb": true,
    "shortName": "reporting_module_shortname",
    "version": "reporting_module_version",
    "vendor": "SQLSERVER",
    "machineName": "localhost",
    "instanceName": "MSSQLSERVER",
    "port": "1433",
    "odbcDriver": "ODBC Driver 17 for SQL Server",
    "ldmSchemaName": "LDM_SCHEMA_NAME",
    "gdmSchemaName": "GDM_SCHEMA_NAME",
    "gdmApproach": "NewGdm"
}
```

3.3.1.7 Response

Name	Description	Schema
workspaceId	**The created RE workspace ID**	**Long**
jobId	**The id of the process that creates the workspace**	**Long**

3.3.2 Workspace status

```
GET /admin-api/external/v1_0/workspaces/${workspaceId}/status
```

3.3.2.1 Request

Name	Located in	Required	Description	Default	Schema
workspaceId	**path**	**yes**	**Workspace ID**	**-**	**Long**

3.3.2.2 Response

Name	Description	Schema
id	**Workspace ID**	**Long**
actualState	**Workspace state**	**String**

```
{
    "id": 1,
    "actualState": "INACTIVE"
}
```

3.3.3 Upgrade workspace

```
PUT /admin-api/external/v1_0/
workspaces/${workspaceId}/upgrade?reportingModuleId=${reportingModuleId}
```

3.3.3.1 Request

Name	Located in	Required	Description	Default	Schema
workspaceId	**path**	**yes**	**Workspace ID**	**-**	**Long**
reportingModuleId	**Path**	**yes**	**Reporting Module ID**	**-**	**Long**
administratorUsername	**Body**	**no**	**Privileged database user (only for MS SQL Server).**	**-**	**String**
administratorPassword	**Body**	**No**	**Privileged database user password (only for MS SQL Server).**	**-**	**String**
gdmSchemaPassword	**Body**	**No**	**GDM user password (only for Oracle)**	**-**	**String**

Name	Located in	Required	Description	Default	Schema
ldmSchemaPassword	**Body**	**No**	**LDM user password (only for Oracle)**	**-**	**String**

3.3.3.2 Example: upgrade GDM LDM workspace on Oracle

```
{
  "gdmSchemaPassword": "GDM_PASSWORD",
  "ldmSchemaPassword": "LDM_PASSWORD"
}
```

3.3.3.3 Example: upgrade LDM only workspace on Oracle

```
{
  "ldmSchemaPassword": "LDM_PASSWORD"
}
```

3.3.3.4 Example: upgrade workspace on MS SQL Server using login/password authentication

```
{
  "administratorPassword": "ADMIN_PASSWORD ",
  "administratorUsername": "ADMIN_USERNAME"
}
```

3.3.3.5 Example: upgrade workspace on MS SQL Server using integrated security

Either empty body should be passed or empty administrator username and password:

```
{

}
```

3.3.3.6 Response

Name	Description	Schema
commandId	**The asynchronous job ID**	**Long**

3.3.4 Upgrade unmanaged workspaces sharing GDM

```
PUT /admin-api/external/v1_0/workspaces/bulkUpgrade
```

3.3.4.1 Request

Name	Located in	Required	Description	Default	Schema
workspaces	Body	yes	Workspace ID	-	Array<Object>, See definition table below
administratorUsername	Body	no	Privileged database user (only for MS SQL Server).	-	String
administratorPassword	Body	No	Privileged database user password (only for MS SQL Server).	-	String
gdmSchemaPassword	Body	No	GDM user password (only for Oracle).	-	String

3.3.4.2 Definition table of nested object in workspaces field

Name	Located in	Required	Description	Default	Schema
workspaceId	workspaces	yes	Workspace ID	-	Long
reportingModuleId	workspaces	yes	Reporting Module ID	-	Long

3.3.4.3 Example: upgrade unmanaged workspaces sharing GDM on Oracle

```
{
  "gdmSchemaPassword": "GDM_PASSWORD",
  "workspaces": [
    {
      "id": 1,
      "reportingModuleId": 1
    },
    {
      "id": 2,
      "reportingModuleId": 2
    }
  ]
}
```

3.3.4.4 Example: upgrade unmanaged workspaces sharing GDM on MS SQL Server using login/password authentication

```
{
  "administratorPassword": "ADMIN_PASSWORD",
  "administratorUsername": "ADMIN_USERNAME",
  "workspaces": [
    {
      "id": 1,
      "reportingModuleId": 1
    },
    {
      "id": 2,
      "reportingModuleId": 2
    }
  ]
}
```

3.3.4.5 Response

Name	Description	Schema
commandId	**The asynchronous job ID**	**Long**

3.3.5 Optimize workspace

```
PUT /admin-api/external/v1_0/workspaces/${workspaceId}/optimize
```

3.3.5.1 Request

Name	Located in	Required	Description	Default	Schema
workspaceId	**path**	**yes**	**Workspace ID**	**-**	**Long**
administratorUsername	**Body**	**no**	**Privileged database user (only for MS SQL Server).**	**-**	**String**
administratorPassword	**Body**	**No**	**Privileged database user password (only for MS SQL Server).**	**-**	**String**
gdmSchemaPassword	**Body**	**No**	**GDM user password (only for Oracle)**	**-**	**String**
ldmSchemaPassword	**Body**	**No**	**LDM user password (only**	**-**	**String**

Name	Located in	Required	Description	Default	Schema
			for Oracle)		
gdmSchemaName	Body	No	GDM username (only for Oracle)	-	String
ldmSchemaName	Body	No	LDM username (only for Oracle)	-	String

3.3.5.2 Example: optimize GDM LDM workspace on Oracle

```
{
  "gdmSchemaName": "GDM_USERNAME",
  "gdmSchemaPassword": "ok",
  "ldmSchemaName": "LDM_USERNAME",
  "ldmSchemaPassword": "ok"
}
```

3.3.5.3 Example: optimize LDM only workspace on Oracle

```
{
  "ldmSchemaName": "LDM_USERNAME",
  "ldmSchemaPassword": "ok"
}
```

3.3.5.4 Example: optimize workspace on MS SQL Server using login/password authentication

```
{
  "administratorPassword": "ADMIN_PASSWORD",
  "administratorUsername": "ADMIN_USERNAME"
}
```

3.3.5.5 Example: optimize workspace on MS SQL Server using integrated security

Either empty body should be passed or empty administrator username and password:

```
{

}
```

3.3.5.6 Response

Name	Description	Schema
commandId	**The asynchronous job ID**	**Long**

3.4 Resource Group API

3.4.1 Upgrade Resource Group

It is possible to upgrade a resource group by adding/removing dataset or submission resources.

```
PUT /admin-api/external/v1_0/resource-groups/${resourceGroupId}
```

This endpoint is intended to be used in combination with dataset/submission creation and retrieval endpoints of rh-api.

3.4.1.1 Request

Name	Located in	Required	Description	Default	Schema
resourceGroupId	**path**	**yes**	**Resource GroupID**	**-**	**Long**
itemId	**Body**	**yes**	**dataset or submission id**	**-**	**Long**
type	**Body**	**yes**	**type (“DATASET” or “SUBMISSION”)**	**-**	**String**
workspaceId	**Body**	**yes**	**workspace id**	**-**	**Long**
selected	**Body**	**yes**	**change selection**	**-**	**Boolean**

Upgrade resource group

```
[
    {
        "itemId": 123,
        "type": "DATASET",
        "workspaceId": 1,
        "selected": true
    },
    {
        "itemId": 234,
        "type": "DATASET",
```

```
            "workspaceId": 1,
            "selected": true
        },
        {
            "itemId": 456,
            "type": "SUBMISSION",
            "workspaceId": 1,
            "selected": true
        }
    ]
```

4 Reporting hub API

4.1 Submission template API

Create submission template

```
POST /rh-api/external/v1_0/submission-templates
```

4.1.1 Request

Name	Located in	Required	Description	Default	Schema
name	**body**	**yes**	**The submission template name**	**-**	**String**
reportingModuleId	**body**	**yes**	**The reporting module ID**	**-**	**Long**
reportingEntityId	**body**	**yes**	**Reporting entity ID**	**-**	**Long**
reportSetId	**body**	**yes**	**Report set ID**	**-**	**Long**
frequency	**body**	**yes**	**Frequency**	**-**	**String**
responsibleUserId	**body**	**no**	**Responsible user ID**	**-**	**String**
deliveryProgramId	**body**	**yes**	**Transmission ID**	**-**	**Long**
regulator	**body**	**yes**	**Regulator**	**-**	**String**
selectedReports	**body**	**yes**	**Array of objects containing report ID, assignee ID and non-mandatory field report extension**	**-**	**Array<Object>, see description table below**

4.1.1.1 Definition table of nested object in selectedReports field

Name	Located in	Required	Description	Default	Schema
reportId	**selectedRe**	**yes**	**The report ID**	**-**	**Long**

Name	Located in	Required	Description	Default	Schema
	ports				
assigneeId	**selectedReports**	**yes**	**The assignee user ID**	**-**	**String**
reportExtension	**selectedReports**	**yes**	**Report extension**	**-**	**String**


```
{
    "deliveryProgramId":  "3",
    "regulator":  "test regulator",
    "reportingModuleId":  "22",
    "frequency":  "MONTHLY",
    "selectedReports":  [
                            {
                                "reportId":  "1",
                                "assigneeId":  "user1",
                                "reportExtension":  "AU"
                            },
                            {
                                "reportId":  "2",
                                "assigneeId":  "",
                                "reportExtension":  "AU"
                            }
                        ],
    "reportingEntityId":  "21",
    "name":  "test",
    "reportSetId":  "1"
}
```


4.1.2 Response

Name	Description	Schema
id	**The submission template ID**	**Long**
name	**The submission template name**	**String**
reportingModule	**The reporting module information**	**Object**
reportingEntity	**Reporting entity information**	**Object**
reportSet	**Report set information**	**Object**
frequency	**Frequency**	**Object**

Name	Description	Schema
responsibleUser	**Responsible user information**	**Object**
deliveryProgram	**Delivery program information**	**Object**
regulator	**Regulator**	**String**
selectedReports	**Array of selected reports**	**Array<Object>**
personInCharge	**Information related to person in charge**	**Object**
createdDate	**Created date**	**Date**


```
{
    "id": 4,
    "name": "test",
    "reportingEntity": {
        "id": "21",
        "name": null,
        "linkedType": "ReportingEntityTO"
    },
    "reportSet": {
        "id": "1",
        "name": null,
        "linkedType": "ReportSetTO"
    },
    "frequency": {
        "name": "Monthly",
        "value": "MONTHLY"
    },
    "reportingModule": {
        "id": "22",
        "name": null,
        "linkedType": "ReportingModuleTO",
        "shortName": null
    },
    "responsibleUser": null,
    "selectedReportCount": 2,
    "totalReportCount": 0,
    "deliveryProgram": {
        "id": "3",
        "name": null,
        "linkedType": "ReportTransmissionTO"
    },
    "createdDate": null,
    "regulator": null,
    "personInCharge": null,
    "selectedReports": [
        {
            "report": {
                "id": "1",
                "name": null,
                "linkedType": "RgTngReportTO",
```

```
                "typeSpecificProperty": "AU"
            },
            "assignee": {
                "id": "user1",
                "name": "user1",
                "linkedType": "UserTO"
            }
        },
        {
            "report": {
                "id": "2",
                "name": null,
                "linkedType": "RgTngReportTO",
                "typeSpecificProperty": "AU"
            },
            "assignee": null
        }
    ]
}
```


4.2 Data set API

4.2.1 Create data set

```
POST /rh-api/external/v1_0/datasets/{workspaceId}
```

4.2.1.1 Request

Name	Located in	Required	Description	Schema
workspaceId	**path**	**yes**	**The workspace ID on which the data set will be created**	**Integer**
dataSetName	**body**	**yes**	**Name of data set**	**String**
dataLot	**body**	**yes**	**ID of data lot**	**Integer**
referenceDate	**body**	**yes**	**Reference date**	**Date**


```
{"dataLot":"1", "dataSetName":"dataSet1", "referenceDate":"2017-03-31"}
```


4.2.1.2 Response

Name	Type	Description
commandId	**Long**	**ID of asynchronous job which was**

Name	Type	Description
		run to trigger data set creation process

4.2.2 Get data sets

```
GET /rh-api/external/v1_0/datasets
```

4.2.2.1 Request

Name	Located in	Required	Description	Schema
offset	**query**	**Yes**	**Offset for retrieving data sets**	**Integer**
size	**query**	**yes**	**Number of data sets to retrieve**	**Integer**

4.2.2.2 Response

Page of data sets:

Name	Type	Description
id	**integer**	**ID of data set**
dataSetName	**string**	**Name of data set**
dataSetType	**string**	**Type of data set**
dataLot	**object**	**Data lot**
dataLot.id	**integer**	**ID of data lot**
referenceDate	**date**	**Reference date**
submissionsAssigned	**integer**	**Number of submissions which were assigned to this data set**
submissionsAllocated	**integer**	**Number of submissions which**

Name	Type	Description
		reports were allocated using this data set
databaseName	**string**	**Name of the database this data set belongs to**
lastModifiedBy	**string**	**ID of the user who last modified the data set**
lastModified	**date-time**	**Last time the data set was modified**


```
{
  "data": {
    "content": [
      {
        "id": 1,
        "dataSetName": "ds1",
        "dataSetType": "ENRICHED",
        "dataLot": {
          "id": 2
          },
        "referenceDate": "2018-01-01",
        "workspace": {
          "id": "21",
          "name": "EA_WS",
          "linkedType": "WorkSpaceTO"
        },
        "submissionsAssigned": 2,
        "submissionsAllocated": 2,
        "databaseName": "EA_WS_LDM"
      }
    ],
    "totalElements": 1,
    "offset": 0,
    "size": 15
  },
  "isDataSetsCreationRunning": false
}
```


4.2.3 Discard data set

```
DELETE /rh-api/external/v1_0/datasets/{workspaceId}/{datasetId}
```

4.2.3.1 Request

Name	Located in	Required	Description	Schema
workspaceId	**path**	**yes**	**The value of the workspace ID of the data set**	**integer**
datasetId	**path**	**yes**	**The value of the data set ID inside workspace**	**integer**
deleteInputData	**query**	**yes**	**If true then also delete the input data of the data set. This parameter will be taken into consideration, only in the scenario which allows this possibility. In other scenarios, if present, it will be silently ignored**	**boolean**
forceDeleteResultData	**query**	**yes**	**If true proceed with deletion of the data set in the case when result set(s) of the data set are linked to submission(s). This parameter will be taken into consideration only in scenarios where it is relevant. In other scenarios, if present, it will be silently ignored**	**boolean**

4.2.3.2 Response

Name	Type	Description
commandId	**Long**	**ID of asynchronous job that was run to trigger data set discard process**

4.2.4 Get data set ID

```
GET/rh-api/external/v1_0/datasets/getid/{workspaceId}/{dataLotId}/{referenceDate}
```

4.2.4.1 Request

Name	Located in	Required	Description	Schema
workspaceId	**path**	**yes**	**The value of the workspace ID of the data set**	**Integer**
dataLotId	**path**	**yes**	**ID of data lot**	**Integer**
referenceDate	**path**	**yes**	**Reference date**	**Date**

4.2.4.2 Response

Name	Type	Description
dataSetId	**integer**	**ID of the data set**

4

4.2.5 Trigger data set input data validation process

```
POST /rh-api/external/v1_0/datasets/{workspaceId}/{datasetId}/input-data/
validation
```

4.2.5.1 Request

Name	Located in	Required	Description	Schema
workspaceId	**path**	**yes**	**The value of the workspace ID of the data set**	**integer**
datasetId	**path**	**yes**	**The value of the data set ID inside workspace**	**integer**

4.2.5.2 Response

Name	Type	Description
commandExecutionId	**integer**	**ID of asynchronous job that was run to trigger validation process**

```
81
```

4.2.6 Retrieve data set input data validation status

```
GET /rh-api/external/v1_0/datasets/{workspaceId}/{datasetId}/input-data/
validation/status
```

4.2.6.1 Request

Name	Located in	Required	Description	Schema
workspaceId	**path**	**yes**	**The value of the workspace ID of the data set**	**integer**
datasetId	**path**	**yes**	**The value of the data set ID inside workspace**	**integer**

4.2.6.2 Response

Name	Type	Description
status	**string**	**The status of input data validation for data set. Possible values:** • **NOT_VALIDATED** • **IN_PROGRESS** • **VALID** • **INVALID**

```
{"status":"VALID","errorsCount":0}
```

4.2.7 Get process parameter set ID

```
GET /rh-api/external/v1_0/processing-params/{workspaceId}
```

4.2.7.1 Request

Name	Located in	Required	Description	Schema
workspaceI	**path**	**yes**	**The value of the workspace ID**	**Long**
enrichmentProcessI	**query**	**yes**	**The value of enrichment process ID**	**String**

4.2.7.2 Response

Name	Type	Description
processParameterSetId	**Long**	**ID of process parameter set**

2

4.2.8 Trigger data set enrichment process

```
POST /rh-api/external/v1_0/datasets/{workspaceId}/{datasetId}/input-data/run-process
```

4.2.8.1 Request

Name	Located in	Required	Description	Schema
workspaceId	**path**	**yes**	**The value of the workspace ID of the data set**	**Long**
datasetId	**path**	**yes**	**The value of the data set ID in the workspace on which enrichment will be triggered**	**Long**
enrichmentd	**body**	**yes**	**ID of process parameter set**	**Long**
processParameter	**body**	**no**	**Array of process parameters**	**Array**

Name	Located in	Required	Description	Schema
			to be overwritten	

```
{"enrichmentId":2}
or
{
    "enrichmentId": 1,
    "processParameters": [
        {
            "paramId": "institution",
            "value": "BLZ10_ANA420"
        },
        {
            "paramId": "application",
            "value": "ACA"
        },
        {
            "paramId": "finrepCurrency",
            "value": "EUR"
        }
    ]
}
```

4.2.8.2 Response

Name	Type	Description
commandId	**Long**	**ID of asynchronous job that was run to trigger data set enrichment process**

5

4.2.9 Get list of result sets

```
GET /rh-api/external/v1_0/datasets/{workspaceId}/{datasetId}/resultsets
```

4.2.9.1 Request

Name	Located in	Required	Description	Schema
workspaceId	**path**	**yes**	**The value of the workspace ID of the data set**	**Long**
datasetId	**path**	**yes**	**The value of the data set ID in the workspace on which**	**Long**

Name	Located in	Required	Description	Schema
			enrichment will be triggered	

4.2.9.2 Response

Page of enriched data sets:

Name	Type	Description
id	**Long**	**Result set ID**
processParameterSet	**object**	**Process parameter set**
reportingEntity	**object**	**Reporting entity of data set**
numberOfDataTables	**Long**	**Number of tables from result set**
created	**date**	**Time when result set was created**
createdby	**String**	**ID of the user who created the result set**
lastModified	**date**	**Last time the result set was modified**
lastModifiedBy	**String**	**ID of the user who last modified the result set**
numberOfSubmissions	**integer**	**The number of submissions that are used in this result set**
assignedSubmissionList	**object**	**A list of submission names that are used in this result set.**
actions	**object**	**A list of actions that can be performed on the result set**
dataSetId	**Long**	**ID of data set**
dataSetName	**String**	**Name of data set**

Name	Type	Description
dataSetReferenceDate	**date**	**Reference date**
lotNumber	**Long**	**Identifier of data lot**
assignedToSubmission	**Boolean**	**Shows whether data set is assigned to a submission or not**
postProcessingStatus	**String**	**The status of post processing**
validationStatus	**String**	**The status of validation**


```
{
  "content": [
  {
    "id":  -154,
    "processParameterSet":  "@{id=-154; name=20080731_1_FRS_1;
linkedType=ProcessTO}",
    "reportingEntity":  "@{id=-5; name=test_name-5; linkedType=ReportingEntityTO}",
    "numberOfDataTables":  8,
    "created":  "2013-02-11T18:54:28",
    "lastModified":  "2017-01-27T08:44:36",
    "lastModifiedBy":  "Test_user",
    "numberOfSubmissions":  1,
    "assignedSubmissionList":  "",
    "actions":  "CHANGE_HISTORY DISCARD",
    "dataSetId":  -1,
    "dataSetName":  "testingDataSet1",
    "dataSetReferenceDate":  "2017-04-30",
    "lotNumber":  -1,
    "postProcessingStatus":  "NOT_IN_PROGRESS",
    "validationStatus":  "INVALID"
  }]
}
```


4.3 Submission API

4.3.1 Create submission

```
POST /rh-api/external/v1_0/submissions/{workspaceId}
```

4.3.1.1 Request

Name	Located in	Required	Description	Default	Schema
workspaceId	**path**	**yes**	**The workspace ID on**	**-**	**Long**

Name	Located in	Required	Description	Default	Schema
			which the submission will be created		
submissionName	body	yes	The submission name	-	String
submissionTemplateId	body	yes	The submission template ID	-	Long
referenceDate	body	yes	Reference date	-	Date
reportingDeadline	body	yes	Reporting deadline	-	Date
submitBy	body	yes	Regulator's deadline	-	Date

```
{"submissionName":"S1", "submissionTemplateId":"1", "referenceDate":"2018-01-01",
"reportingDeadline": "2018-01-01", "submitBy":"2018-01-01"}
```

4.3.1.2 Response

Name	Description	Schema
id	The corresponding command execution ID for the submission creation process	Long

```
81
```

4.3.2 Get submission

```
GET /rh-api/external/v1_0/
submissions/{workspaceId}/{reportingEntityId}/{referenceDate}/{reportSetId}/{submi
ssionName}
```

4.3.2.1 Request

Name	Located in	Required	Description	Default	Schema
workspaceId	path	yes	Workspace ID	-	Long

Name	Located in	Required	Description	Default	Schema
reportingentityId	path	yes	Reporting entity ID	-	Long
referenceDate	path	yes	Reference date	-	Date
reportSetId	path	yes	Report set ID	-	Long
submissionName	Path	Yes	Name of Submission	-	String

4.3.2.2 Response

Name	Description	Schema
id	Submission ID	Long
submissionName	The name of submission	String
reportSetName	The name of the report set	String
financialInstitution	Financial institution	String
frequency	Frequency	String
reportingCreate	The date of creating	Date
referenceDate	Reference date	Date
reportingDeadline	Reporting deadline: internal deadline from submission information	Date
totalReports	The number of reports from submission	integer
errorReports	The number of reports with error from submission	integer
workSpace	The submission's workspace	object

Name	Description	Schema
responsiblePerson	**The name of responsible person**	**String**
allocationStatus	**The status of allocation**	**String**
lastModified	**The last time the submission was modified**	**Date**
submissionStatus	**The status of submission**	**object**
reactivationReason	**The reason of reactivation**	**String**
reactivationUser	**The name of the user who reactivated the submission**	**String**
reactivationDate	**The date when the submission was reactivated**	**Date**


```
{
    "id": 2,
    "submissionName": "dummyReportingSetName Reporting Entity Name",
    "reportSetName": "dummyReportingSetName",
    "financialInstitution": "Reporting Entity Name",
    "frequency": null,
    "reportingCreate": {"dayOfMonth": 7, "dayOfWeek": "FRIDAY","month":
"JUNE","year": 2019, "monthValue": 6, "hour": 16, "minute": 7, "nano": 0,
"second": 18, "dayOfYear": 158, "chronology": { "id": "ISO","calendarType":
"iso8601"}},
    "referenceDate": {"year": 2021, "month": "AUGUST","dayOfMonth": 22,
"dayOfWeek": "SUNDAY","era": "CE","monthValue": 8, "dayOfYear": 234, "leapYear":
false, "chronology": {"id": "ISO","calendarType": "iso8601"}},
    "reportingDeadline": {"year": 2021, "month": "AUGUST","dayOfMonth": 22,
"dayOfWeek": "SUNDAY","era": "CE","monthValue": 8, "dayOfYear": 234, "leapYear":
false, "chronology": {"id": "ISO","calendarType": "iso8601" }},
    "submitBy": {"year": 2021, "month": "AUGUST","dayOfMonth": 22, "dayOfWeek":
"SUNDAY","era": "CE","monthValue": 8, "dayOfYear": 234, "leapYear": false,
"chronology": {"id": "ISO","calendarType": "iso8601"}},
    "totalReports": 2,
    "errorReports": 0,
    "workspace": {"id": "10005","name": "it-unique-dummy-workspace-
name","linkedType": "WorkSpaceTO"},
    "responsiblePerson": "John Doe",
    "allocationStatus": null,
    "lastModified": {"dayOfMonth": 7, "dayOfWeek": "FRIDAY","month":
"JUNE","year": 2019, "monthValue": 6, "hour": 13, "minute": 7, "nano": 0,
"second": 18, "dayOfYear": 158, "chronology": {"id": "ISO","calendarType":
"iso8601"}},
    "submissionStatus": null,
    "reactivationReason": null,
    "reactivationUser": null,
    "reactivationDate": null
}
```

4.3.3 Discard submission

```
DELETE /rh-api/external/v1_0/submissions/{workspaceId}/{submissionId}
```

4.3.3.1 Request

Name	Located in	Required	Description	Default	Schema
workspaceId	**path**	**yes**	**Workspace ID**	**-**	**integer (int64)**
submissionId	**path**	**yes**	**Submission ID**	**-**	**integer (int64)**

4.3.3.2 Response

Name	Description	Schema
id	**The corresponding command execution ID for the submission discard process**	**Long**

4.3.4 Assign result sets to submission

```
POST /rh-api/external/v1_0/submissions/{workspaceId}/{submissionId}/resultsets
```

4.3.4.1 Request

Name	Located in	Required	Description	Default	Schema
workspaceId	**path**	**yes**	**Workspace ID**	**-**	**integer (int64)**
submissionId	**path**	**yes**	**Submission ID**	**-**	**integer (int64)**
resultSetIds	**body**	**yes**	**A list that contains the result sets IDs**	**-**	**Array<Long>**
selectAll	**body**	**yes**	**If all result sets are**	**-**	**Boolean**

Name	Located in	Required	Description	Default	Schema
			selected or not		

```
{" resultSetIds":[1,2,3], "selectAll":true}
```

4.3.4.2 Response

No data

4.3.5 Count assigned result sets

```
GET /rh-api/external/v1_0/submissions/{workspaceId}/{submissionId}/resultsets/
count
```

4.3.5.1 Request

Name	Located in	Required	Description	Default	Schema
workspaceId	**path**	**yes**	**Workspace ID**	**-**	**Long**
submissionId	**path**	**yes**	**Submission ID**	**-**	**Long**

4.3.5.2 Response

Name	Description	Schema
count	**The count of result sets which are assigned to submission**	**integer**

```
{"count":"3"}
```

4.3.6 Get post process ids by report set ids

```
GET /rh-api/external/v1_0/submissions/{workspaceId}/{submissionId}/post-processing
/getByIds
```

4.3.6.1 Request

Name	Located in	Required	Description	Default	Schema
workspaceId	**path**	**yes**	**Workspace ID**	**-**	**Long**
submissionId	**path**	**yes**	**Submission ID**	**-**	**Long**
reportSetId	**query**	**yes**	**A comma separated string representing report set IDs**	**-**	**String**

4.3.6.2 Response

```
{[1,2,3]}
```

4.3.7 Postprocess

```
POST /rh-api/external/v1_0/submissions/{workspaceId}/{submissionId}/post-processing
```

4.3.7.1 Request

Name	Located in	Required	Description	Default	Schema
workspaceId	**path**	**yes**	**Workspace ID**	**-**	**Long**
submissionId	**path**	**yes**	**Submission ID**	**-**	**Long**
reportSetPostProcessingId	**query**	**yes**	**reportSetPostProcessing ID**	**-**	**Long**

4.3.7.2 Response

Name	Type	Description
commandId	**Long**	**ID of asynchronous job which was run to trigger post processing process.**

81

4.3.8 Trigger submission allocation process

```
POST /rh-api/external/v1_0/submissions/{workspaceId}/{submissionId}/allocation
```

4.3.8.1 Request

Name	Located in	Required	Description	Default	Schema
submissionId	**path**	**yes**	**Submission ID**	**-**	**String**
workspaceId	**path**	**yes**	**Workspace ID**	**-**	**Long**
preserveManual	**query**	**no**	**Preserve manual**	**-**	**Boolean**

4.3.8.2 Response

Name	Description	Schema
commandId	**The corresponding command execution ID for allocation process**	**Long**

81

4.3.9 Trigger submission validation process

```
POST /rh-api/external/v1_0/submissions/{workspaceId}/{submissionId}/validation
```

4.3.9.1 Request

Name	Located in	Required	Description	Default	Schema
submissionId	**path**	**yes**	**Submission ID**	**-**	**string**
workspaceId	**path**	**yes**	**Workspace ID**	**-**	**Long**

4.3.9.2 Response

Name	Description	Schema
commandId	**The corresponding command execution ID for validation process**	**Long**

81

4.3.10 Trigger delivery process

```
POST /rh-api/external/v1_0/submissions/{workspaceId}/{submissionId}/transmission
```

4.3.10.1 Request

Name	Located in	Required	Description	Default	Schema
workspaceId	**path**	**yes**	**Workspace ID**	**-**	**Long**
submissionId	**path**	**yes**	**Submission ID**	**-**	**Long**

4.3.10.2 Response

Name	Description	Schema
commandId	**The corresponding command execution ID for delivery process**	**Long**

81

4.3.11 Get transmission artifacts

```
GET /rh-api/external/v1_0/submissions/{workspaceId}/{submissionId}/transmission
```

4.3.11.1 Request

Name	Located in	Required	Description	Default	Schema
workspaceId	**path**	**yes**	**Workspace ID**	**-**	**Long**
submissionId	**path**	**yes**	**Submission ID**	**-**	**Long**

4.3.11.2 Response

Name	Description	Schema
name	**The name of the artifact**	**String**
extension	**The extension of the artifact**	**String**
path	**The path for the artifact**	**String**
Count	**Number of artifacts**	**Integer**


```
{
"value" : [
   {
     "name": "transmissionFile1",
     "extension": "xml",
     "path": "submissions/-1/46/transmission/transmissionFile1.xml"
   }
 ],
"Count":1
}
```


4.3.12 Get transmission file

```
GET /rh-api/external/v1_0/
submissions/{workspaceId}/{submissionId}/transmission/{transmissionFile}
```

4.3.12.1 Request

Name	Located in	Required	Description	Default	Schema
workspaceId	**path**	**yes**	**Workspace ID**	**-**	**Long**

Name	Located in	Required	Description	Default	Schema
submissionId	**path**	**yes**	**Submission ID**	**-**	**Long**
trancmissionFileName	**Path**	**Yes**	**The name of transmission file**	**-**	**String**

4.3.12.2 Response

Name	Description	Schema
transmissionFile	**A transmission file**	**File**

4.3.13 Add submission references to submission

```
POST /rh-api/external/v1_0/submissions/{workspaceId}/{submissionId}/references
```

4.3.13.1 Request

Name	Located in	Required	Description	Default	Schema
workspaceId	**path**	**yes**	**Workspace ID**	**-**	**Long**
submissionId	**path**	**yes**	**Submission ID**	**-**	**Long**
reportingSetId	**Body**	**Yes**	**Id of the reporting set**	**-**	**Long**
reportingSetReferenceName	**Body**	**Yes**	**Name of the reference**	**-**	**String**
submissionInformationId	**Body**	**Yes**	**Id of the submission to reference**	**-**	**Long**

4.3.13.2 Response

Name	Description	Schema
id	**id of the reference**	**Long**

Name	Description	Schema
name	**Reference name**	**String**
description	**Reference description**	**String**
linkedSubmission	**Linked submission**	**Object**

4.3.14 Get submission references

```
GET /rh-api/external/v1_0/submissions/{workspaceId}/{submissionId}/references
```

4.3.14.1 Request

Name	Located in	Required	Description	Default	Schema
workspaceId	**path**	**yes**	**Workspace ID**	**-**	**Long**
submissionId	**path**	**yes**	**Submission ID**	**-**	**Long**

4.3.14.2 Response

Name	Description	Schema
id	**ID of the reference**	**Long**
name	**Reference Name**	**String**
submissionReferences	**Submission references**	**Array**

4.3.15 Get available submissions

4.3.15.1 Request

Name	Located in	Required	Description	Default	Schema
workspaceId	**path**	**yes**	**Workspace ID**	**-**	**Long**

Name	Located in	Required	Description	Default	Schema
submissionId	**path**	**yes**	**Submission ID**	**-**	**Long**
offset	**path**	**yes**	**Offset**	**-**	**Long**
size	**path**	**yes**	**Number of items from offset**	**-**	**Long**
reportingEntity	**path**	**No**	**Reporting entity name**	**-**	**String**
reportSetId	**path**	**No**	**Report set id**	**-**	**Long**
referenceDate	**path**	**No**	**Reference date**	**-**	**Date**

4.3.15.2 Response

List of available submissions, an object having as “content” an array of:

Name	Description	Schema
id	**ID of the submission**	**Long**
submissionInformationId	**Submission information id**	**Long**
submissionReferenceId	**Submission reference id**	**Long**
referenceDate	**Reference date**	**Date**
submissionName	**Submission name**	**String**
reportSetName	**Report set name**	**String**
financialInstitution	**Name of financial institution**	**String**
status	**Status**	**“ACTIVE”/ “ARCHIVED” / “ARCHIVED_AS_SCENARIO”**
responsiblePerson	**Responsible person**	**String**

4.3.16 Remove submission references from submission

```
DELETE /rh-api/external/v1_0/
submissions/{workspaceId}/{submissionId}/references/{submissionReferenceId}
```

4.3.16.1 Request

Name	Located in	Required	Description	Default	Schema
workspaceId	**path**	**yes**	**Workspace ID**	**-**	**Long**
submissionId	**path**	**yes**	**Submission ID**	**-**	**Long**
submissionReferenceId	**Body**	**Yes**	**Submission reference ID**	**-**	**Long**

4.3.16.2 Response

HTTP status

4.4 Job API

4.4.1 Get job status

```
GET /rh-api/external/v1_0/processes/status
```

4.4.1.1 Request

Name	Located in	Required	Description	Default	Schema
jobId	**query**	**yes**	**CommandExec ID**	**-**	**Long**

4.4.1.2 Response

Name	Description	Schema
status	**The status of the command**	**String**

{"COMPLETED"}

4.4.2 Get job status (recommeded)

```
GET /rh-api/external/v1_0/processes/job-status
```

4.4.2.1 Request

Name	Located in	Required	Description	Default	Schema
jobId	**query**	**yes**	**CommandExec ID**	**-**	**Long**

4.4.2.2 Response

Name	Description	Schema
status	**The status of the command**	**String**

```
{"status":"COMPLETED"}
```

4.5 Bridge structure API

4.5.1 Prepare data structure to launch bridge

```
PUT /rh-api/external/v1_0/
submissions/{workspaceId}/{submissionId}/databridge/{callerStringUniqueIdentifier}
```

4.5.1.1 Request

Name	Located in	Required	Description	Default	Schema
submissionId	**path**	**yes**	**Submission ID**	**-**	**Long**
workspaceId	**path**	**yes**	**Workspace ID**	**-**	**Long**
callerStringUniqueIdentifier	**path**	**yes**	**A unique string identifying the external component performing the data load**	**-**	**String**

4.5.1.2 Response

Name	Description	Schema
dataSetId	**The created RE data set ID**	**Long**
inputDataSetId	**The created RE input data set ID**	**Long**
resultDataSetId	**The created RE result data set ID**	**Long**
rgTngDataSetId	**The created RG TNG data set ID**	**Long**

4.5.2 Prepare data structure to launch bridge with multiple data units

```
PUT /rh-api/external/v1_0/
submissions/{workspaceId}/{submissionId}/databridge/{callerStringUniqueIdentifier}
/{requestedNumberOfDataUnits}
```

4.5.2.1 Request for one or more data units

Name	Located in	Required	Description	Default	Schema
submissionId	**path**	**yes**	**Submission ID**	**-**	**Long**
workspaceId	**path**	**yes**	**Workspace ID**	**-**	**Long**
callerStringUniqueIdentifier	**path**	**yes**	**Unique string identifying the external component performing the data load**	**-**	**String**
requestedNumberOfDataUnits	**Path**	**Optional**	**The number of RG TNG data units required to load its data**	**1**	**Long**

4.5.2.2 Response

Name	Description	Schema
dataSetId	**The created RE data set ID**	**Long**

Name	Description	Schema
inputDataSetId	**The created RE input data set ID**	**Long**
resultDataSetId	**The created RE result data set ID**	**Long**
rgTngDataSetId	**The created RG TNG data set ID**	**Long**
rgTngDataUnits	**Array of RG TNG data unit IDs**	**Array<Long>**

```
{
  "dataSetId": 1,
  "inputDataSetId": 2,
  "resultDataSetId": 3,
  "rgTngDataContainers":
  {
   "rgTngDataSetId" :4,
   "rgTngDataUnits":[5,6,7]
  }
}
```

5 Admin Center PowerShell script samples

5.1 Utility Scripts

5.1.1 Constants

Constants.ps1 contains message texts and asynchronous operations' timeout values.

```
Write-Host "Constants initialized"

$failed = "FAILED"
$success = "SUCCESS"

# messages
$createWorkspaceSuccessMessage = "Creation of workspace was successful"
$createWorkspaceFailMessage = "Creation of workspace failed"
$optimizeWorkspaceSuccessMessage = "Workspace optimization triggered"
$optimizeWorkspaceFailMessage = "Could not trigger workspace optimization"
$upgradeWorkspaceSuccessMessage = "Workspace ugrade triggered"
$upgradeWorkspaceFailMessage = "Could not trigger workspace upgrade"

$workspaceOperationSuccessMessage = "Workspace operation successfully finished"
$workspaceOperationFailureMessage = "Workspace operation has failed"

$jobTimeout = 2400
$pollInterval = 20
$iterations = [int]($jobTimeout / $pollInterval)
```

5.1.2 Lookup workspace status

The script *lookup_workspace_status.ps1* This sample script is used to search status for created workspace.

```
#Script checks the current status of workspace

param (
    [parameter(Position = 1, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $workspaceId,
    [parameter(Position = 2, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $securityToken,
    [parameter(Position = 3, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $reportingBaseUri
)
. $PSScriptRoot\Constants.ps1

#create the cookie using the security token received from input parameters, you
should not change it
$session = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$cookie = New-Object System.Net.Cookie
$cookie.Name = "__Secure-WKFS-Token"
```

```
$cookie.Value = "$securityToken"

$uriObject = [System.Uri]"$reportingBaseUri"
$cookie.Domain = $uriObject.Host
$cookie.Path = $uriObject.AbsolutePath

#add the cookie to the session object
$session.Cookies.Add($cookie);

$resultObj = New-Object -TypeName psobject
$failed = "FAILED"
$success = "SUCCESS"

$commandSuccessStatuses = "INACTIVE", "NEEDS_CONFIGURATION", "ACTIVE"
$commandEndStatuses = "INACTIVE", "NEEDS_CONFIGURATION", "IMPORT_FAILED",
"INSTALL_FAILED", "REPORTING_MODULE_PATCH_FAILED", "SIBLING_OPERATION_FAILED",
"ENGINE_UPGRADE_FAILED", "ENGINE_UPGRADE_NEEDED",
"STANDALONE_ENGINE_UPGRADE_FAILED"
$commandStatus = "STARTING", "STANDALONE_ENGINE_UPGRADE_RUNNING"
$maxIteration = 1

################## Workspace status lookup #########################

$baseApiUrl = $reportingBaseUri + '/external/v1_0'
$APIMethod = "workspaces/" + $workspaceId + "/status"

try
{
    while ((-Not($commandEndStatuses -contains $commandStatus)) -and
($maxIteration -lt $iterations))
    {
        $response = Invoke-RestMethod  "$( $baseApiUrl )/$( $APIMethod )" -Method
Get -ContentType "application/json" -WebSession $session | ConvertTo-Json
        $responseJson = ConvertFrom-Json $response
        $workspaceStatus = $responseJson.actualState
        Write-Host "Workspace status.. $workspaceStatus "
        if ($commandSuccessStatuses -contains $workspaceStatus)
        {
            $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$success
            $resultObj | Add-Member -MemberType NoteProperty -Name actualState
-Value $workspaceStatus
            Write-Host "Workspace operation was successful"
            break
        }
        if ($workspaceStatus -like '*FAILED*')
        {
            Write-Host "Workspace operation has failed"
            Write-Host $workspaceStatus
            $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$failed
            break
        }

        $finished = $commandEndStatuses -contains $workspaceStatus

        if (-Not$finished)
        {
            Write-Host "Wait $pollInterval seconds and try again.."
            start-sleep -seconds $pollInterval
        }
        $maxIteration = $maxIteration + 1
```

```
    }

    Write-Host "lookup_workspace_status script finished"
}
catch [System.Net.WebException]
{
    $resultObj | Add-Member -MemberType NoteProperty -Name status -Value $failed
    Write-Host "Script run failed"
    Write-Host $( $_.Exception )
    Write-Host "An exception was caught: $( $_.Exception.Message )"
}

return $resultObj
```

5.2 Reporting module uploading script

This sample is used to upload a reporting module. It includes another request that checks the upload status. The request that checks the status of operation is posted every 20 seconds until the uploading is finished or more than 2400 seconds have passed. When the script has finished successfully, the reporting module is uploaded and return own ID.

```
#Script uploads a reporting module

param (
    [parameter(Position = 1, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $reportingModulePath,
    [parameter(Position = 2, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $reportingModuleShortName,
    [parameter(Position = 3, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $reportingModuleVersion,
    [parameter(Position = 4, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $securityToken,
    [parameter(Position = 5, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $reportingBaseUri
)
. $PSScriptRoot\Constants.ps1

#create the cookie using the security token received from input parameters, you
should not change it
$session = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$cookie = New-Object System.Net.Cookie
$cookie.Name = "__Secure-WKFS-Token"
$cookie.Value = "$securityToken"

$uriObject = [System.Uri]"$reportingBaseUri"
$cookie.Domain = $uriObject.Host
$cookie.Path = $uriObject.AbsolutePath

#add the cookie to the session object
$session.Cookies.Add($cookie);
```

```
$resultObj = New-Object -TypeName psobject

$commandSuccessStatus = "FINISHED"
$commandEndStatuses = $commandSuccessStatus, "FAILED", "CANCELLED", "CANCELLING"
$commandStatus = "RUNNING"
$maxIteration = 1
$failed = "FAILED"
$success = "SUCCESS"

################## Reporting module upload ##################

$baseApiUrl = $reportingBaseUri + '/external/v1_0'
$APIMethod = "package-manager/modules"

try
{
    Write-Host "Call rest endpoint to trigger the upload reporting module rest
endpoint"
    Write-Host $baseApiUrl + $APIMethod
    Write-Host $requestBody
    Write-Host $cookie
    Write-Host $session

$fileBytes = [System.IO.File]::ReadAllBytes($reportingModulePath);
$fileEnc =
[System.Text.Encoding]::GetEncoding("ISO-8859-1").GetString($fileBytes);
$boundary = [System.Guid]::NewGuid().ToString();

Write-Host "boundary - $boundary"
$LF = "`r`n";
$URL = "$( $baseApiUrl )/$( $APIMethod )"
$filename = $(Split-Path -Leaf -Path $reportingModulePath)

$bodyLines = (
    "--$boundary",
    "Content-Disposition: form-data; name=`"file`"; filename=`"$filename`"",
    "Content-Type: application/octet-stream$LF",
    $fileEnc,
    "--$boundary--$LF"
) -join $LF

$response = Invoke-RestMethod -Uri $URL -Method Post -ContentType "multipart/form-
data; boundary=`"$boundary`"" -Body $bodyLines -WebSession $session | ConvertTo-
Json
}
catch [System.Net.WebException]
{
    Write-Host "An exception was caught: $( $_.Exception.Message )"
    $resultObj | Add-Member -MemberType NoteProperty -Name uploadMessage -Value
$failed
    throw "Reproting module upload failed"
}

try
{
    ################## Checking the job status ##################

    $responseJson = ConvertFrom-Json $response
    $jobId = $responseJson.status
    Write-Host "Job ID is.. $jobId"
```

```
    $baseApiUrl = $reportingBaseUri + '/external/v1_0'
    $APIMethod = "jobs/" + $jobId

    while ((-Not($commandEndStatuses -contains $commandStatus)) -and
($maxIteration -lt $iterations))
    {
        Write-Host "Running try $( $maxIteration ) out of max $iterations or until
successful"
        Write-Host "Call rest end point: $( $baseApiUrl )/$( $APIMethod ) to
retrieve the uploading job status"
        $commandStatus = Invoke-WebRequest "$( $baseApiUrl )/$( $APIMethod )"
-Method GET -WebSession $session
        if ($commandStatus -like '*'+$commandSuccessStatus+'*')
        {
            Write-Host "Reporting module upload process was finished successful"
            $resultObj | Add-Member -MemberType NoteProperty -Name uploadStatus
-Value $success
            break
        }
        if ($commandStatus -like '*FAILED*')
        {
            break
        }

        $finished = $commandEndStatuses -contains $commandStatus

        if (-Not$finished)
        {
            Write-Host "As the process status is not in a final state we will wait
$pollInterval seconds and try again"
            start-sleep -seconds $pollInterval
        }
        $maxIteration = $maxIteration + 1
    }

    if ($commandStatus -eq "RUNNING") {
        Write-Host "The uploading process is still in progress you might want to
increase the number of attempts"
        break
    }

    if (-Not($commandStatus -like '*'+$commandSuccessStatus+'*')){
        Write-Host "The uploading process of this file was failed"
        $resultObj | Add-Member -MemberType NoteProperty -Name jobStatus -Value
$failed
    }
}
 catch [System.Net.WebException]
{
    Write-Host "An exception was caught: $( $_.Exception.Message )"
    $resultObj | Add-Member -MemberType NoteProperty -Name jobMessage -Value
$failed
    throw "Checking the job status failed"
}

try
{
    ################## Retrieving the reporting module id ##################

    $baseApiUrl = $reportingBaseUri + '/external/v1_0/package-manager'
    $APIMethod = "status?shortName=" + $reportingModuleShortName + "&version=" +
$reportingModuleVersion
```

```
    Write-Host "Call rest endpoint $( $baseApiUrl )/$( $APIMethod ) to get the
Reporting module id"
    $response = Invoke-RestMethod  "$( $baseApiUrl )/$( $APIMethod )" -Method Get
-ContentType "application/json" -WebSession $session | ConvertTo-Json

    if ($response)
    {
        Write-Host "Retrieval was successful"
        $resultObj | Add-Member -MemberType NoteProperty -Name reportingModuleId
-Value $response
        Write-Host "ID of uploaded Reporing module is... " $response
    }
    else
    {
        Write-Host "Retrieval was failed"
        $resultObj | Add-Member -MemberType NoteProperty -Name idRetrievalStatus
-Value $failed
        throw "Retrieval ID of Reporting module is failed!"
    }
}
catch [System.Net.WebException]
{
    Write-Host "An exception was caught: $( $_.Exception.Message )"
    $resultObj | Add-Member -MemberType NoteProperty -Name idRetrievalMessage
-Value $failed
    throw "Retrieving the reporting module id failed"
}
return $resultObj
```

5.3 Workspace creation script

This sample is used for the creation of a workspace entity. It includes another request that checks the workspace status. The request that checks the status of operation is posted every 20 seconds until the creation is finished or more than 2400 seconds have passed. When the script has finished successfully, the workspace has been created.

```
#This sample is used for the creation of a workspace

#specific input parameters for workspace creation
param (
    [parameter(Position = 1, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $workspaceName,
    [parameter(Position = 2, Mandatory = $false, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $description,
    [parameter(Position = 3, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $workspaceType,
    [parameter(Position = 4, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $targetApplication,
    [parameter(Position = 5, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $newDb,
    [parameter(Position = 6, Mandatory = $false, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $shortName,
    [parameter(Position = 7, Mandatory = $false, ValueFromPipeline = $true,
```

```
ValueFromRemainingArguments = $false)]
    $version,
    [parameter(Position = 8, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $vendor,
    [parameter(Position = 9, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $machineName,
    [parameter(Position = 10, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $serviceName,
    [parameter(Position = 11, Mandatory = $false, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $tnsServiceName,
    [parameter(Position = 12, Mandatory = $false, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $instanceName,
    [parameter(Position = 13, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $port,
    [parameter(Position = 14, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $odbcDriver,
    [parameter(Position = 15, Mandatory = $false, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $locationLink,
    [parameter(Position = 16, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $administratorPassword,
    [parameter(Position = 17, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $administratorUsername,
    [parameter(Position = 18, Mandatory = $false, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $gdmSchemaName,
    [parameter(Position = 19, Mandatory = $false, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $gdmSchemaPassword,
    [parameter(Position = 20, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $ldmSchemaName,
    [parameter(Position = 21, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $ldmSchemaPassword,
    [parameter(Position = 22, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $gdmApproach,
    [parameter(Position = 23, Mandatory = $false, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $connectionUsername,
    [parameter(Position = 24, Mandatory = $false, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $connectionPassword,
    [parameter(Position = 25, Mandatory = $false, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $sharedWorkspaces,
    [parameter(Position = 26, Mandatory = $false, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $sharedRODirectoryRoot,
    [parameter(Position = 27, Mandatory = $false, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $newSharedRODirectoryRoot,
```

```
    [parameter(Position = 28, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $securityToken,
    [parameter(Position = 29, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $reportingBaseUri,
    [parameter(Position = 30, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $reportingModulePath
);
. $PSScriptRoot\Constants.ps1

################## Utils ##################
function assertOperationResult($operationResult,$successMessage,$failMessage) {

    if ($operationResult.status -eq $failed){
        throw $failMessage
    } else {
        Write-Host $successMessage
    }
}

#create the cookie using the security token received from input parameters, you
should not change it
$session = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$cookie = New-Object System.Net.Cookie
$cookie.Name = "__Secure-WKFS-Token"
$cookie.Value = "$securityToken"
$failed = "FAILED"
$success = "SUCCESS"

$uriObject = [System.Uri]"$reportingBaseUri"
$cookie.Domain = $uriObject.Host
$cookie.Path = $uriObject.AbsolutePath

#add the cookie to the session object
$session.Cookies.Add($cookie);

$reportingModuleUploadSuccessMessage = "Reporting Module upload operation
successfully finished"
$reportingModuleUploadFailureMessage = "Reporting Module upload operation has
failed"

    Write-Host "-------------------------------------------------------"
    Write-Host "Process started"
    Write-Host "-------------------------------------------------------"


    ########### Lookup uploaded module ID ###########
    $lookupUploadedModuleResult = & "$PSScriptRoot\
upload_reporting_module_script_with_job_status.ps1" -reportingModulePath
$reportingModulePath -reportingModuleShortName $shortName -reportingModuleVersion
$version -securityToken $securityToken -reportingBaseUri $reportingBaseUri
    assertOperationResult -operationResult $lookupUploadedModuleResult
-successMessage $reportingModuleUploadSuccessMessage -failMessage
$reportingModuleUploadFailureMessage


    Write-Host "-------------------------------------------------------"
```

```
    Write-Host Reporting module ID is
$lookupUploadedModuleResult.reportingModuleId
    Write-Host "-------------------------------------------------------"

################## Workspace creation ##################

#create the api url for workspace creation
#'admin-api' is common for all available functionalities, you should not change it
$baseApiUrl = $reportingBaseUri + '/external/v1_0'
$APIMethod = 'workspaces'

#create the object from given input data  and convert it to json
$requestParameters = @{
    name = "$workspaceName"
    description = "$description"
    workspaceType = "$workspaceType"
    targetApplication = "$targetApplication"
    newDb = "$newDb"
    shortName = "$shortName"
    version = "$version"
    vendor = "$vendor"
    machineName = "$machineName"
    serviceName = "$serviceName"
    tnsServiceName = "$tnsServiceName"
    instanceName = "$instanceName"
    port = "$port"
    odbcDriver = "$odbcDriver"
    locationLink = "$locationLink"
    administratorPassword = "$administratorPassword"
    administratorUsername = "$administratorUsername"
    gdmSchemaName = "$gdmSchemaName"
    gdmSchemaPassword = "$gdmSchemaPassword"
    ldmSchemaName = "$ldmSchemaName"
    ldmSchemaPassword = "$ldmSchemaPassword"
    gdmApproach = "$gdmApproach"
    connectionUsername = "$connectionUsername"
    connectionPassword = "$connectionPassword"
    sharedWorkspaces = "$sharedWorkspaces"
    sharedRODirectoryRoot = "$sharedRODirectoryRoot"
    newSharedRODirectoryRoot = "$newSharedRODirectoryRoot"
}

$requestBody = ConvertTo-Json $requestParameters
#call the rest endpoint for workspace creation and verify that the response
contains workspace id
$resultObj = New-Object -TypeName psobject
try
{
    Write-Host "Call the workspace creation rest endpoint"
    Write-Host $baseApiUrl
    Write-Host $securityToken
    Write-Host $requestBody
    Write-Host $cookie
    Write-Host $session
    $response = Invoke-WebRequest  "$( $baseApiUrl )/$( $APIMethod )" -Body
$requestBody -Method Post -ContentType "application/json" -WebSession $session
    if ($response.StatusCode -eq 201)
    {
        $responseJson = ConvertFrom-Json $response
        if ($responseJson)
        {
            Write-Host $createWorkspaceSuccessMessage
```

```
            $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$success
            $resultObj | Add-Member -MemberType NoteProperty -Name id -Value
$responseJson


            ########### Lookup workspace installation status ###########
            $lookupWorkspaceInstallationStatus = & "$PSScriptRoot\
lookup_workspace_status.ps1" -workspaceId $responseJson -securityToken
$securityToken -reportingBaseUri $reportingBaseUri
            assertOperationResult -operationResult
$lookupWorkspaceInstallationStatus -successMessage
$workspaceOperationSuccessMessage -failMessage $workspaceOperationFailureMessage
        }
    }
    else
    {
        Write-Host $response
        Write-Host $createWorkspaceFailMessage
        $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$failed
    }
}
catch [System.Net.WebException]
{
    Write-Host $createWorkspaceFailMessage
    Write-Host "An exception was caught: $( $_.Exception.Message )"
    $resultObj | Add-Member -MemberType NoteProperty -Name status -Value $failed
    throw "Workspace creation failed"
}

return $resultObj
```

5.4 Workspace upgrading script

This sample is used to upgrade the workspace. It includes another request that checks the workspace status. The request that checks the status of operation is posted every 20 seconds until the creation is finished or more than 2400 seconds have passed. When the script has finished successfully, the workspace has been upgraded.

```
#This sample is used to upgrade workspace

#specific input parameters for workspace upgrade
param (
    [parameter(Position = 1, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $workspaceId,
    [parameter(Position = 2, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $reportingModuleId,
    [parameter(Position = 3, Mandatory = $false, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $administratorUsername,
    [parameter(Position = 4, Mandatory = $false, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $administratorPassword,
    [parameter(Position = 5, Mandatory = $false, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
```

```
    $ldmSchemaPassword,
    [parameter(Position = 6, Mandatory = $false, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $gdmSchemaPassword,
    [parameter(Position = 7, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()] $securityToken,
    [parameter(Position = 8, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()] $reportingBaseUri
);
. $PSScriptRoot\Constants.ps1

################## Utils ##################
function assertOperationResult($operationResult,$successMessage,$failMessage) {

    if ($operationResult.status -eq $failed){
        throw $failMessage
    } else {
        Write-Host $successMessage
    }
}

#create the cookie using the security token received from input parameters, you
should not change it
$session = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$cookie = New-Object System.Net.Cookie
$cookie.Name = "__Secure-WKFS-Token"
$cookie.Value = "$securityToken"
$failed = "FAILED"
$success = "SUCCESS"

$uriObject = [System.Uri]"$reportingBaseUri"
$cookie.Domain = $uriObject.Host
$cookie.Path = $uriObject.AbsolutePath

#add the cookie to the session object
$session.Cookies.Add($cookie);


################## Workspace upgrade ##################

#create the api url for workspace upgrade
#'admin-api' is common for all available functionalities, you should not change it
$baseApiUrl = $reportingBaseUri + '/external/v1_0'
$APIMethod = 'workspaces/' + $workspaceId + '/upgrade?reportingModuleId=' +
$reportingModuleId

#create the object from given input data  and convert it to json
$requestParameters = @{
    administratorUsername = "$administratorUsername"
    administratorPassword = "$administratorPassword"
    ldmSchemaPassword = "$ldmSchemaPassword"
    gdmSchemaPassword = "$gdmSchemaPassword"
}

$requestBody = ConvertTo-Json $requestParameters
#call the rest endpoint for workspace upgrade and verify that the response
contains job id
$resultObj = New-Object -TypeName psobject
try
{
```

```
    Write-Host "Call the workspace upgrade rest endpoint"
    Write-Host $baseApiUrl + $APIMethod
    Write-Host $securityToken
    Write-Host $requestBody
    Write-Host $cookie
    Write-Host $session
    $response = Invoke-WebRequest  "$( $baseApiUrl )/$( $APIMethod )" -Body
$requestBody -Method Put -ContentType "application/json" -WebSession $session
    if ($response.StatusCode -eq 200)
    {
        $responseJson = ConvertFrom-Json $response
        if ($responseJson)
        {
            Write-Host $upgradeWorkspaceSuccessMessage
            $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$success
            $resultObj | Add-Member -MemberType NoteProperty -Name id -Value
$responseJson

            ########### Workspace lookup ###########
            $lookupWorkspaceInstallationStatus = & "$PSScriptRoot\
lookup_workspace_status.ps1" -workspaceId $workspaceId -securityToken
$securityToken -reportingBaseUri $reportingBaseUri
            assertOperationResult -operationResult
$lookupWorkspaceInstallationStatus -successMessage
$workspaceOperationSuccessMessage -failMessage $workspaceOperationFailureMessage
        }
    }
    else
    {
        Write-Host $response
        Write-Host $upgradeWorkspaceFailMessage
        $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$failed
    }
}
catch [System.Net.WebException]
{
    Write-Host $upgradeWorkspaceFailMessage
    Write-Host "An exception was caught: $( $_.Exception.Message )"
    $resultObj | Add-Member -MemberType NoteProperty -Name status -Value $failed
    throw "Workspace upgrade failed"
}

return $resultObj
```

5.5 Multiple unmanaged workspaces sharing the same GDM upgrading script

This sample is used to upgrade two or more unmanaged workspaces sharing the sam GDM. It includes another request that checks the workspace status. The request that checks the status of operation is posted every 20 seconds until the creation is finished or more than 2400 seconds have passed. When the script has finished successfully, the workspaces has been upgraded.

```
#This sample is used to upgrade workspace
#specific input parameters for workspace upgrade
param (
    [parameter(Position = 1, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
```

```
    [ValidateNotNullOrEmpty()]
    $workspaces,
    [parameter(Position = 2, Mandatory = $false, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $administratorUsername,
    [parameter(Position = 3, Mandatory = $false, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $administratorPassword,
    [parameter(Position = 4, Mandatory = $false, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $ldmSchemaPassword,
    [parameter(Position = 5, Mandatory = $false, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $gdmSchemaPassword,
    [parameter(Position = 6, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()] $securityToken,
    [parameter(Position = 7, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()] $reportingBaseUri
);
. $PSScriptRoot\Constants.ps1


################## Utils ##################
function assertOperationResult($operationResult,$successMessage,$failMessage) {

    if ($operationResult.status -eq $failed){
        throw $failMessage
    } else {
        Write-Host $successMessage
    }
}

# function accepts String argument in format
"workspace1,reportingModule1;workspace2,reportingmodule2"
# and converts into array of hashtables
# ouput example (after conversion into json):
# [
#  {
#   id: 1,
#   reportingModuleId: 1
#  },
#  {
#   id: 2,
#   reportingModuleId: 2
#  }
# ]
function ParseWorkspaces($Text)
{
    $workspaces = @()
    foreach ($item in $Text.Split(";"))
    {
        if ($item.split(",")[0])
        {
            $workspace = @{ }
            $workspace.add('id', $item.split(",")[0]);
            $workspace.add('reportingModuleId', $item.split(",")[1]);
            $workspaces += $workspace
        }
    }
    return $workspaces
```

```
}
#create the cookie using the security token received from input parameters, you
should not change it
$session = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$cookie = New-Object System.Net.Cookie
$cookie.Name = "__Secure-WKFS-Token"
$cookie.Value = "$securityToken"
$failed = "FAILED"
$success = "SUCCESS"

$uriObject = [System.Uri]"$reportingBaseUri"
$cookie.Domain = $uriObject.Host
$cookie.Path = $uriObject.AbsolutePath

#add the cookie to the session object
$session.Cookies.Add($cookie);

################## Multiple workspaces upgrade ##################

#create the api url for workspace upgrade
#'admin-api' is common for all available functionalities, you should not change it
$baseApiUrl = $reportingBaseUri + '/external/v1_0'
$APIMethod = 'workspaces/bulkUpgrade'

#convert workspaces string into json array
$workspaces = @(ParseWorkspaces -Text $workspaces)

$workspaceId = $($workspaces[0].Values)[0]
#create the object from given input data  and convert it to json
$requestParameters = @{
    administratorUsername = "$administratorUsername"
    administratorPassword = "$administratorPassword"
    ldmSchemaPassword = "$ldmSchemaPassword"
    gdmSchemaPassword = "$gdmSchemaPassword"
    workspaces = $workspaces
}

$requestBody = ConvertTo-Json $requestParameters
#call the rest endpoint for workspace upgrade and verify that the response
contains job id
$resultObj = New-Object -TypeName psobject
try
{
    Write-Host "Call the multiple workspaces upgrade rest endpoint"
    Write-Host $baseApiUrl + $APIMethod
    Write-Host $securityToken
    Write-Host $requestBody
    Write-Host $cookie
    Write-Host $session
    $response = Invoke-WebRequest  "$( $baseApiUrl )/$( $APIMethod )" -Body
$requestBody -Method Put -ContentType "application/json" -WebSession $session
    if ($response.StatusCode -eq 200)
    {
        $responseJson = ConvertFrom-Json $response
        if ($responseJson)
        {
            Write-Host $upgradeWorkspaceSuccessMessage
            $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$success
            $resultObj | Add-Member -MemberType NoteProperty -Name id -Value
$responseJson
```

```
            ########### Workspace lookup ###########
            $lookupWorkspaceInstallationStatus = & "$PSScriptRoot\
lookup_workspace_status.ps1" -workspaceId $workspaceId -securityToken
$securityToken -reportingBaseUri $reportingBaseUri
            assertOperationResult -operationResult
$lookupWorkspaceInstallationStatus -successMessage
$workspaceOperationSuccessMessage -failMessage $workspaceOperationFailureMessage
        }
    }
    else
    {
        Write-Host $response
        Write-Host $upgradeWorkspaceFailMessage
        $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$failed
    }
}
catch [System.Net.WebException]
{
    Write-Host $upgradeWorkspaceFailMessage
    Write-Host "An exception was caught: $( $_.Exception.Message )"
    $resultObj | Add-Member -MemberType NoteProperty -Name status -Value $failed
    throw "Workspace upgrade failed"
}

return $resultObj
```

5.6 Workspace optimization script

This sample is used to optimize the workspace. It includes another request that checks the workspace status. The request that checks the status of operation is posted every 20 seconds until the creation is finished or more than 2400 seconds have passed. When the script has finished successfully, the workspace has been optimized.

```
#This sample is used to optimize workspace

#specific input parameters for workspace optimization
param (
    [parameter(Position = 1, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $workspaceId,
    [parameter(Position = 2, Mandatory = $false, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $administratorUsername,
    [parameter(Position = 3, Mandatory = $false, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $administratorPassword,
    [parameter(Position = 4, Mandatory = $false, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $ldmSchemaName,
    [parameter(Position = 5, Mandatory = $false, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $ldmSchemaPassword,
    [parameter(Position = 6, Mandatory = $false, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $gdmSchemaName,
    [parameter(Position = 7, Mandatory = $false, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $gdmSchemaPassword,
```

```
    [parameter(Position = 8, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()] $securityToken,
    [parameter(Position = 9, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()] $reportingBaseUri
);
. $PSScriptRoot\Constants.ps1

################## Utils ##################
function assertOperationResult($operationResult,$successMessage,$failMessage) {

    if ($operationResult.status -eq $failed){
        throw $failMessage
    } else {
        Write-Host $successMessage
    }
}

#create the cookie using the security token received from input parameters, you
should not change it
$session = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$cookie = New-Object System.Net.Cookie
$cookie.Name = "__Secure-WKFS-Token"
$cookie.Value = "$securityToken"
$failed = "FAILED"
$success = "SUCCESS"

$uriObject = [System.Uri]"$reportingBaseUri"
$cookie.Domain = $uriObject.Host
$cookie.Path = $uriObject.AbsolutePath

#add the cookie to the session object
$session.Cookies.Add($cookie);

################## Workspace optimization ##################

#create the api url for workspace optimization
#'admin-api' is common for all available functionalities, you should not change it
$baseApiUrl = $reportingBaseUri + '/external/v1_0'
$APIMethod = 'workspaces/' + $workspaceId + '/optimize'

#create the object from given input data  and convert it to json
$requestParameters = @{
    administratorUsername = "$administratorUsername"
    administratorPassword = "$administratorPassword"
    ldmSchemaName = "$ldmSchemaName"
    ldmSchemaPassword = "$ldmSchemaPassword"
    gdmSchemaName = "$gdmSchemaName"
    gdmSchemaPassword = "$gdmSchemaPassword"
}

$requestBody = ConvertTo-Json $requestParameters
#call the rest endpoint for workspace optimization and verify that the response
contains job id
$resultObj = New-Object -TypeName psobject
try
{
    Write-Host "Call the workspace optimization rest endpoint"
    Write-Host $baseApiUrl + $APIMethod
    Write-Host $securityToken
    Write-Host $requestBody
```

```
    Write-Host $cookie
    Write-Host $session
    $response = Invoke-WebRequest  "$( $baseApiUrl )/$( $APIMethod )" -Body
$requestBody -Method Put -ContentType "application/json" -WebSession $session
    if ($response.StatusCode -eq 200)
    {
        $responseJson = ConvertFrom-Json $response
        if ($responseJson)
        {
            Write-Host $optimizeWorkspaceSuccessMessage
            $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$success
            $resultObj | Add-Member -MemberType NoteProperty -Name id -Value
$responseJson

            ########### Workspace lookup ###########
            $lookupWorkspaceInstallationStatus = & "$PSScriptRoot\
lookup_workspace_status.ps1" -workspaceId $workspaceId -securityToken
$securityToken -reportingBaseUri $reportingBaseUri
            assertOperationResult -operationResult
$lookupWorkspaceInstallationStatus -successMessage
$workspaceOperationSuccessMessage -failMessage $workspaceOperationFailureMessage
        }
    }
    else
    {
        Write-Host $response

#create the cookie using the security token received from input parameters, you
should not change it
$session = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$cookie = New-Object System.Net.Cookie
$cookie.Name = "__Secure-WKFS-Token"
$cookie.Value = "$securityToken"
$failed = "FAILED"
$success = "SUCCESS"

$uriObject = [System.Uri]"$reportingBaseUri"
$cookie.Domain = $uriObject.Host
$cookie.Path = $uriObject.AbsolutePath

#add the cookie to the session object
$session.Cookies.Add($cookie);


################## Workspace optimization ##################

#create the api url for workspace optimization
#'admin-api' is common for all available functionalities, you should not change it
$baseApiUrl = $reportingBaseUri + '/external/v1_0'
$APIMethod = 'workspaces/' + $workspaceId + '/optimize'

#create the object from given input data  and convert it to json
$requestParameters = @{
    administratorUsername = "$administratorUsername"
    administratorPassword = "$administratorPassword"
    ldmSchemaName = "$ldmSchemaName"
    ldmSchemaPassword = "$ldmSchemaPassword"
    gdmSchemaName = "$gdmSchemaName"
    gdmSchemaPassword = "$gdmSchemaPassword"
}
```

```
$requestBody = ConvertTo-Json $requestParameters
#call the rest endpoint for workspace optimization and verify that the response
contains job id
$resultObj = New-Object -TypeName psobject
try
{
    Write-Host "Call the workspace optimization rest endpoint"
    Write-Host $baseApiUrl + $APIMethod
    Write-Host $securityToken
    Write-Host $requestBody
    Write-Host $cookie
    Write-Host $session
    $response = Invoke-WebRequest  "$( $baseApiUrl )/$( $APIMethod )" -Body
$requestBody -Method Put -ContentType "application/json" -WebSession $session
    if ($response.StatusCode -eq 200)
    {
        $responseJson = ConvertFrom-Json $response
        if ($responseJson)
        {
            Write-Host $optimizeWorkspaceSuccessMessage
            $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$success
            $resultObj | Add-Member -MemberType NoteProperty -Name id -Value
$responseJson

            ########### Workspace lookup ###########
            $lookupWorkspaceInstallationStatus = & "$PSScriptRoot\
lookup_workspace_status.ps1" -workspaceId $workspaceId -securityToken
$securityToken -reportingBaseUri $reportingBaseUri
            assertOperationResult -operationResult
$lookupWorkspaceInstallationStatus -successMessage
$workspaceOperationSuccessMessage -failMessage $workspaceOperationFailureMessage
        }
    }
    else
    {
        Write-Host $response
        Write-Host $optimizeWorkspaceFailMessage
        $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$failed
    }
}
catch [System.Net.WebException]
{
    Write-Host $optimizeWorkspaceFailMessage
    Write-Host "An exception was caught: $( $_.Exception.Message )"
    $resultObj | Add-Member -MemberType NoteProperty -Name status -Value $failed
    throw "Workspace optimization failed"
}

return $resultObj
```

5.7 Resource Group update script

```
. $PSScriptRoot\Constants.ps1

#create the cookie for RH app
$session = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$cookie = New-Object System.Net.Cookie
$cookie.Name = "__Secure-WKFS-Token"
```

```
$cookie.Value = "tokenvalue"

$reportingBaseUri = "https://dev-regapp-d01.wkfs-frc.local:8081/reporting-webapp"
$uriObject = [System.Uri]"$reportingBaseUri"
$cookie.Domain = $uriObject.Host
$cookie.Path = $uriObject.AbsolutePath
$session.Cookies.Add($cookie);

#create the cookie for admin app
$session2 = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$cookie2 = New-Object System.Net.Cookie
$cookie2.Name = "__Secure-WKFS-Token"
$cookie2.Value = " tokenvalue "

$adminBaseUri = "https://dev-regapp-d01.wkfs-frc.local:8081/admin-webapp"
$uriObject2 = [System.Uri]"$adminBaseUri"
$cookie2.Domain = $uriObject2.Host
$cookie2.Path = $uriObject2.AbsolutePath
$session2.Cookies.Add($cookie2);

$resultObj = New-Object -TypeName psobject

################## Data set creation + assigning it to Resource Group
##################

$workspaceId = "21"
$dataSetName = "ECB Data Set X"
$dataLot = "3"
$referenceDate = "2019-03-31"
$baseApiUrl = $reportingBaseUri + '/rh-api/external/v1_0'
$baseApiUrlAdmin = $adminBaseUri + '/admin-api/external/v1_0'
$APIMethod = "datasets/" + $workspaceId
$APIMethod2 = "datasets/getid/" + $workspaceId + "/" + $dataLot + "/" +
$referenceDate
$resourceGroupId = "1000"
$APIMethod3 = "resource-groups/" + $resourceGroupId
$jobStatusUrl = $baseApiUrl + "/processes"
$commandSuccessStatus = "COMPLETED"
$commandEndStatuses = $commandSuccessStatus, "STOPPED", "FAILED", "ABANDONED"

#create the object(using the input parameters) needed for the request body and
convert it to json
$requestParameters = @{
    dataSetName = "$dataSetName"
    dataLot = "$dataLot"
    referenceDate = "$referenceDate"
}

$requestBody = $requestParameters | ConvertTo-Json
#call the rest endpoint for dataset creation and get the response

try
{
    Write-Host "Call the rest endpoint for dataset creation with parameters
dataSetName: " $dataSetName "  dataLot: " $dataLot " referenceDate: "
$referenceDate
    $datasetCreateJobIdResponse = Invoke-RestMethod -SkipCertificateCheck "$(
$baseApiUrl )/$( $APIMethod )" -Body $requestBody -Method Post -ContentType
"application/json" -WebSession $session | ConvertTo-Json
    Write-Host "Wait 30 seconds for dataset creation"
    start-sleep -seconds 30
```

```
    $failed = "FAILED"
    $success = "SUCCESS"
    $maxIteration = 1
    $finished = $FALSE

    while ((-Not($finished)) -and ($maxIteration -lt $iterations))
    {
        Write-Host "Running try $( $maxIteration ) out of max $iterations or until
successful"

        $APIMethod = "status?jobId=" + $datasetCreateJobIdResponse
        Write-Host "Call the rest endpoint for the job status of the creation job:
" $datasetCreateJobIdResponse
        $jobStatusResponse = Invoke-WebRequest -SkipCertificateCheck "$(
$jobStatusUrl )/$( $APIMethod )" -Method GET -WebSession $session
        Write-Host "Job status is: "$jobStatusResponse
        if ($jobStatusResponse -like '*'+$commandSuccessStatus+'*')
        {
            $resultObj | Add-Member -MemberType NoteProperty -Name id -Value
$parsedCommandDetailsResponse.id -Force
            $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$success -Force
            $finished = $TRUE
            Write-Host "Dataset creation confirmed"

            $datasetId = Invoke-WebRequest -SkipCertificateCheck "$( $baseApiUrl
)/$( $APIMethod2 )" -Method GET -WebSession $session

            Write-Host "Dataset ID: " $datasetId

            $updateRequestParameters = @(
            @{
                itemId = "$datasetId"
                type = "DATASET"
                workspaceId = "$workspaceId"
                selected = "true"
            })
            $updateRequestBody = ConvertTo-Json $updateRequestParameters

            Write-Host "Dataset added to resource group."

            $Response = Invoke-RestMethod -SkipCertificateCheck "$(
$baseApiUrlAdmin )/$( $APIMethod3 )" -Body $updateRequestBody -Method Put
-ContentType "application/json" -WebSession $session2 | ConvertTo-Json
        }
        if ($commandStatus -like '*FAILED*')
        {
            $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$failed -Force
            throw "Creation of data set failed!"
        }
        $finished = $commandEndStatuses -contains $jobStatusResponse

        if (-Not$finished)
        {
            start-sleep -seconds $pollInterval
            Write-Host "Wait $pollInterval seconds and try again.."
        }
        $maxIteration = $maxIteration + 1

        $maxIteration = $maxIteration + 1
    }
```

```
}
catch [System.Net.WebException]
{
    $resultObj | Add-Member -MemberType NoteProperty -Name status -Value $failed
-Force
    Write-Host "Dataset creation failed"

    Write-Host "An exception was caught: $($_.Exception.Message)"
    $result = $_.Exception.Response.GetResponseStream()
    $reader = New-Object System.IO.StreamReader($result)
    $reader.BaseStream.Position = 0
    $reader.DiscardBufferedData()
    $responseBody = $reader.ReadToEnd();
    $_.Exception.Response
}

return $resultObj
```

6 Reporting Hub PowerShell script samples

6.1 Main script

The following PowerShell sample script contains an end-to-end test. The business flow iterates through a considerable number of important functionalities of the application, i.e.:

- Create submission template
- Add data set
- Lookup data set ID
- Validate input data
- Enrich input data
- Lookup created result sets
- Add submission
- Lookup submission ID
- Add result set to submission
- Postprocess submission
- Allocate reports
- Validate reports
- Create transmission file

For each functionality, a separate script is created that is used in the main script but can also be run independently. These scripts contain several chunks of code that can be reused for other functionalities such as creating a cookie. The cookie is created using the security token received from the input parameter (you should not change it) and it is required to be sent with each request as Wolters Kluwer Token cookie.

```
param (
    [parameter(Position = 1, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $securityToken ,
    [parameter(Position = 2, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $reportingBaseUri
)

################## Imports ##################
. $PSScriptRoot\ProcessParameters.ps1
. $PSScriptRoot\Constants.ps1

################## Utils ##################
function assertOperationResult($operationResult,$successMessage,$failMessage) {

    if ($operationResult.status -eq $failed){
        throw $failMessage
    } else {
```

```
        Write-Host $successMessage
    }
}

################ Main ###############
try
{
    Write-Host "-------------------------------------------------------"
    Write-Host "Process started"
    Write-Host "-------------------------------------------------------"
     ########## Submission template creation ###########
    $submissionTemplateCreationResult = & "$PSScriptRoot\
external_create_submission_template.ps1" -submissionTemplateName
$submissionTemplateName -reportingEntityId $reportingEntityId -reportingModuleId
$reportingModuleId -regulator $regulator -reportSetId $reportingSet
-deliveryProgramId $deliveryProgramId -frequency $frequency -selectedReports
$selectedReports -securityToken $securityToken -reportingBaseUri $reportingBaseUri
    assertOperationResult -operationResult $submissionTemplateCreationResult
-successMessage $createSubmissionTemplateSuccessMessage -failMessage
$createSubmissionTemplateFailMessage
    $templateId = $submissionTemplateCreationResult.id

    ########### Data set creation ###########
    $dataSetCreationResult = & "$PSScriptRoot\external_create_data_set.ps1"
-dataSetName $dataSetName -dataLot $dataLot -referenceDate $referenceDate
-workspaceId $workspaceId -securityToken $securityToken -reportingBaseUri
$reportingBaseUri
    assertOperationResult -operationResult $dataSetCreationResult -successMessage
$dataSetSuccessCreationMessage -failMessage $dataSetFailedCreationMessage

    ########### Lookup dataset ID ###########
    $lookupDatasetIdResult = & "$PSScriptRoot\external_lookup_data_set_id.ps1"
-dataLot $dataLot -referenceDate $secondReferenceDate -workspaceId $workspaceId
-securityToken $securityToken -reportingBaseUri $reportingBaseUri
    assertOperationResult -operationResult $lookupDatasetIdResult -successMessage
$dataSetSuccessfulIdLookupMessage -failMessage $dataSetFailedIdLookupMessage

    ########### Validate input data ###########
    $validateInputDataResult = & "$PSScriptRoot\
external_validate_input_data_set.ps1" -dataSetId $lookupDatasetIdResult.id
-workspaceId $workspaceId -securityToken
$securityToken -reportingBaseUri $reportingBaseUri
    assertOperationResult -operationResult $validateInputDataResult
-successMessage $dataSetSuccessValidationMessage -failMessage
$dataSetFailedValidationMessage

    ########### Enrich input data ###########
    $enrichInputDataResult = & "$PSScriptRoot\external_enrich_input_data_set.ps1"
-dataSetId $lookupDatasetIdResult.id -enrichmentId $enrichmentId -workspaceId
$workspaceId -securityToken $securityToken -reportingBaseUri $reportingBaseUri
    assertOperationResult -operationResult $enrichInputDataResult -successMessage
$enrichmentOfInputDataSuccessMessage -failMessage
$enrichmentOfInputDataFailedMessage

    ########### Lookup created result sets ###########
    $lookupCreatedResultSetsResult = & "$PSScriptRoot\
external_lookup_create_result_set.ps1" -dataSetId $lookupDatasetIdResult.id
-workspaceId $workspaceId
-securityToken $securityToken -reportingBaseUri $reportingBaseUri
    assertOperationResult -operationResult $lookupCreatedResultSetsResult
-successMessage $lookupCreatedResultSetsSuccessMessage -failMessage
$lookupCreatedResultSetsFailedMessage
```

```
if ($lookupCreatedResultSetsResult.id.Count -eq 0) {
Write-Host "No result sets were created during the enrichment. Exiting script as
there is nothing left to do."
exit -1
}
    ########### Create submission ###########
    $createSubmissionResult = & "$PSScriptRoot\external_create_submission.ps1"
-templateId $templateId  -submissionName $submissionName
-submissionReferenceDate $submissionReferenceDate -reportingDeadline
$reportingDeadline
 -submitBy $submitBy -workspaceId $workspaceId -securityToken $securityToken
-reportingBaseUri $reportingBaseUri
    assertOperationResult -operationResult $createSubmissionResult -successMessage
$createSubmissionSuccessMessage -failMessage $createSubmissionFailMessage

    ########### Lookup submission ID ###########
    $lookupSubmissionIdResult = & "$PSScriptRoot\
external_lookup_submission_id.ps1" -reportingSet $reportingSet -reportingDate
$reportingDate -reportingEntity
$reportingEntity -workspaceId $workspaceId -securityToken $securityToken
-reportingBaseUri $reportingBaseUri
    assertOperationResult -operationResult $lookupSubmissionIdResult
-successMessage $submissionIdLookupSuccessMessage -failMessage
$submissionIdLookupFailMessage

    ########### Add data set to submission ###########
    $listOfIds = @()
    $listOfIdsSeparatedByComma = ""
    foreach ($element in $lookupCreatedResultSetsResult.id) {
        $listOfIds += $element.id
        $listOfIdsSeparatedByComma = $listOfIdsSeparatedByComma + $element.id +
","
    }

Write-Host "List of result sets Ids that will be added to submission: $listOfIds"

   $addDataSetToSubmissionResult = & "$PSScriptRoot\
external_add_data_set_to_submission.ps1" -resultSetIds $listOfIds -selectAll
$selectAll -submissionId
$lookupSubmissionIdResult.id -workspaceId $workspaceId -securityToken
$securityToken -reportingBaseUri $reportingBaseUri
   assertOperationResult -operationResult $addDataSetToSubmissionResult
-successMessage $addDataSetToSubmissionSuccessMessage -failMessage
$addDataSetToSubmissionFailMessage

    ########### Postprocess submission ###########

if (($postProcessesToExecute) -and ($postProcessesToExecute.count -gt 0)) {
$lookupPostprocess = & "$PSScriptRoot\lookup_result_set_post_process.ps1"
-submissionId $lookupSubmissionIdResult.id -workspaceId $workspaceId
-securityToken $securityToken -reportingBaseUri $reportingBaseUri
    assertOperationResult -operationResult $lookupPostprocess -successMessage
$lookupResultSetPostProcessSuccessMessage -failMessage
$lookupResultSetPostProcessFailMessage

    $listOfPostProcessIds = $lookupPostprocess.id
Write-Host "Available post-processes: $($listOfPostProcessIds)"

foreach ($postProcessToExecute in $postProcessesToExecute) {
$postProcessId
foreach ($availablePostProcess in $listOfPostProcessIds) {
```

```
if ($availablePostProcess.name -eq $postProcessToExecute) {
$foundPostProcessId = $availablePostProcess.id
break
}
}

if ($foundPostProcessId) {
  Write-Host "Executing post process with id:
  $foundPostProcessId"


  ########### Postprocess submission ###########
  $postprocessSubmissionResult = & "$PSScriptRoot\
external_postprocess_submission.ps1" -submissionId $lookupSubmissionIdResult.id
-workspaceId $workspaceId -reportSetPostProcessingId $foundPostProcessId
-securityToken $securityToken -reportingBaseUri $reportingBaseUri
  assertOperationResult -operationResult $postprocessSubmissionResult
-successMessage $postprocessSubmissionSuccessMessage -failMessage
  $postprocessSubmissionFailMessage

} else {
Write-Host "No post process with name $postProcessToExecute was available for
current submission. Skipping..."
}
}
    }
     ########### Allocate reports ###########
    $allocateReportsResult = & "$PSScriptRoot\external_allocate_report.ps1"
-preserveManual $preserveManual -submissionId $lookupSubmissionIdResult.id
-workspaceId $workspaceId -securityToken $securityToken -reportingBaseUri
$reportingBaseUri
    assertOperationResult -operationResult $allocateReportsResult -successMessage
$allocateReportsSuccessMessage -failMessage $allocateReportsFailMessage

    ########### Validate reports ###########
    $validateReportsResult = & "$PSScriptRoot\external_validate_reports.ps1"
-submissionId $lookupSubmissionIdResult.id -workspaceId $workspaceId
-securityToken $securityToken -reportingBaseUri $reportingBaseUri
    assertOperationResult -operationResult $validateReportsResult -successMessage
$validateReportsSuccessMessage -failMessage $validateReportsFailMessage

    ########### Create transmission file ###########
    $createTransmissionFileResult = & "$PSScriptRoot\
external_create_transmission_file.ps1" -submissionId $lookupSubmissionIdResult.id
-workspaceId $workspaceId
-securityToken $securityToken -reportingBaseUri $reportingBaseUri
    assertOperationResult -operationResult $createTransmissionFileResult
-successMessage $createTransmissionFileSuccessMessage -failMessage
$createTransmissionFileFailMessage

    ########### Download transmission file ###########
    $downloadTransmissionFileResult = & "$PSScriptRoot\
external_download_transmission_file.ps1" -submissionId
$lookupSubmissionIdResult.id -workspaceId $workspaceId
-securityToken $securityToken -reportingBaseUri $reportingBaseUri
    assertOperationResult -operationResult $downloadTransmissionFileResult
-successMessage $downloadTransmissionFileSuccessMessage -failMessage
$downloadTransmissionFileFailMessage

}
catch [System.Net.WebException] {
    Write-Host "An exception was caught: $($_.Exception.Message)"
```

```
}
```

6.2 Utility scripts

In the main script, two utility files are used:

- *ProcessParameters.ps1*
- *Constants.ps.1*

6.2.1 ProcessParameters

The first script, *ProcessParameters.ps1* , is used to define all parameters. When using this script, you only have to enter the data once in the file and all the entries are retrieved from this script. This means you don't have to enter all the parameters in the command line.

```
#Script used to populate most of the parameters used by the other scripts

Write-Host "Process parameters initialized"

$dataSetName = "DataSetNameOne"
$dataLot = "1"
$referenceDate = "2017-03-31"
$secondReferenceDate = "2017-03-31"
$workspaceId = "-1" # ID from WORKSPACE
$reportingBaseUri = "http://localhost:8080/reporting-webapp"
$enrichmentId = "ONE" # ENRICHMENT_ID from ENRICHMENTS - SGMAS_WK_Vocab_ProcessLot
$submissionName = "Custom name"
$reportingDeadline = "2021-04-28"
$submissionReferenceDate = "2021-04-28"
$submitBy = "2021-04-28"

$reportingSet = "-1" # ID from REPORT_SETS - MAS_610_Revised_SGOP
$reportingDate = "2021-04-28" # Needs to coincide with internalDeadline
$reportingEntity = 10001 # ID from REPORTING_ENTITY - Entity1

$selectAll = "false"
$preserveManual = "false"

# properties to create submission template
$submissionTemplateName = "test submission template"
$reportingEntityId = "10001"
$reportingModuleId = "-1"
$regulator = "test regulator"
$deliveryProgramId = "10001"
$frequency = "MONTHLY"
$selectedReports = "30001,user1,AU;"

#$postProcessesToExecute = @("SGMAS_REG_Vocab_ProcessLot")

# Not used
$requestedNumberOfDataUnits = "1"
$callerStringUniqueIdentifier = "SGA_FDA_LDM_DataBridge"
```

6.2.2 Constants

The second script, *Constants.ps.1*, holds all the success and fail messages that are used after running each

script, and that are displayed accordingly in the PowerShell log. This approach makes the main script easier to read.

```
Write-Host "Constants initialized"

$failed = "FAILED"
$success = "SUCCESS"

# messages
$dataSetSuccessCreationMessage = "Dataset created with success"
$dataSetFailedCreationMessage = "Dataset creation failed"

$dataSetSuccessfulIdLookupMessage = "Dataset ID lookup was successful"
$dataSetFailedIdLookupMessage = "Dataset ID lookup failed"

$dataSetSuccessValidationMessage = "Dataset validation was successful"
$dataSetFailedValidationMessage = "Dataset validation failed"

$enrichmentProcessIdLookupSuccessMessage = "Enrichment process ID lookup was
successful"
$enrichmentProcessIdLookupFailedMessage = "Enrichment process ID lookup failed"

$enrichmentOfInputDataSuccessMessage = "Enrichment of input data was successful"
$enrichmentOfInputDataFailedMessage = "Enrichment of input data failed"

$lookupCreatedResultSetsSuccessMessage = "Lookup of created result sets was
successful"
$lookupCreatedResultSetsFailedMessage = "Lookup of created result sets failed"

$createSubmissionTemplateSuccessMessage = "Creation of submission was successful"
$createSubmissionTemplateFailMessage = "Creation of submission failed"

$createSubmissionSuccessMessage = "Creation of submission template was successful"
$createSubmissionFailMessage = "Creation of submission template failed"

$submissionIdLookupSuccessMessage = "Lookup of submission ID was successful"
$submissionIdLookupFailMessage = "Lookup of submission ID failed"

$addDataSetToSubmissionSuccessMessage = "Adding of data sets to submission was
successful"
$addDataSetToSubmissionFailMessage = "Adding of data sets to submission failed"

$lookupResultSetPostProcessSuccessMessage = "Lookup of result sets for post
process was successful"
$lookupResultSetPostProcessFailMessage = "Lookup of result sets for post process
failed"

$postprocessSubmissionSuccessMessage = "Postprocess of submission was successful"
$postprocessSubmissionFailMessage = "Postprocess of submission failed"

$allocateReportsSuccessMessage = "Allocation of reports was successful"
$allocateReportsFailMessage = "Allocation of reports failed"

$validateReportsSuccessMessage = "Validation of reports was successful"
$validateReportsFailMessage = "Validation of reports failed"

$createTransmissionFileSuccessMessage = "Creation of transmission file was
successful"
$createTransmissionFileFailMessage = "Creation of transmission file failed"

$downloadTransmissionFileSuccessMessage = "Download of transmission file was
successful"
```

```
$downloadTransmissionFileFailMessage = "Download of transmission file failed"
```

6.3 Create submission template script

This sample is used for the creation of submission template entity.

```
#This sample is used for the creation of a submission template entity

#specific input parameters for submission template creation
param (
    [parameter(Position = 1, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()] $submissionTemplateName,
    [parameter(Position = 2, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()] $reportingEntityId,
    [parameter(Position = 3, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()] $reportingModuleId,
    [parameter(Position = 4, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()] $regulator,
    [parameter(Position = 5, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()] $reportSetId,
    [parameter(Position = 6, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()] $deliveryProgramId,
    [parameter(Position = 7, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()][ValidateSet('DAILY', 'WEEKLY', 'BI_WEEKLY',
'MONTHLY', 'QUARTERLY', 'YEARLY')] $frequency,
    [parameter(Position = 9, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()] $selectedReports,
    [parameter(Position = 8, Mandatory = $false, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $responsibleUserId,
    [parameter(Position = 9, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()] $securityToken,
    [parameter(Position = 10, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()] $reportingBaseUri
);
. $PSScriptRoot\Constants.ps1

# function accepts String argument in format
"reportId1,assigneeId1,reportExtension1;reportId2,assigneeId2,reportExtension2"
# and converts into array of hashtables
# ouput example (after conversion into json):
# [
#  {
#   reportId: 1,
#   assigneeId: "user",
#   reportExtension: ""
#  },
#  ...
# ]
function ParseSelectedReports($Text)
```

```
{
    $submissionReports = @()
    foreach ($item in $Text.Split(";"))
    {
        if ($item.split(",")[0])
        {
            $submissionReport = @{ }
            $submissionReport.add('reportId', $item.split(",")[0]);
            $submissionReport.add('assigneeId', $item.split(",")[1]);
            $submissionReport.add('reportExtension', $item.split(",")[2]);
            $submissionReports += $submissionReport
        }
    }
    return $submissionReports
}

#create the cookie using the security token received from input parameters, you
should not change it
$session = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$cookie = New-Object System.Net.Cookie
$cookie.Name = "__Secure-WKFS-Token"
$cookie.Value = "$securityToken"
$failed = "FAILED"
$success = "SUCCESS"

$uriObject = [System.Uri]"$reportingBaseUri"
$cookie.Domain = $uriObject.Host
$cookie.Path = $uriObject.AbsolutePath

#add the cookie to the session object
$session.Cookies.Add($cookie);

################## Submission template creation ##################

#create the api url for submission template creation
#'rh-api' is common for all available functionalities, you should not change it
$baseApiUrl = $reportingBaseUri + '/rh-api/external/v1_0/submission-templates'
#transform the selectedReports object (from input parameters)
$selectedReports = @(ParseSelectedReports -Text $selectedReports)

#create the object from given input data and transformed selectedReports object
needed for the request body and convert it to json
$requestParameters = @{
    name = "$submissionTemplateName"
    reportingEntityId = "$reportingEntityId"
    reportSetId = "$reportSetId"
    reportingModuleId = "$reportingModuleId"
    frequency = "$frequency"
    responsibleUserId = "$responsibleUserId"
    deliveryProgramId = "$deliveryProgramId"
    regulator = "$regulator"
    selectedReports = $selectedReports
}

$requestBody = ConvertTo-Json $requestParameters
#call the rest endpoint for submission template creation and verify that the
response contains submission template id
$resultObj = New-Object -TypeName psobject
try
{
    Write-Host "Call the submission template creation rest endpoint"
    $response = Invoke-WebRequest  "$( $baseApiUrl )/$( $APIMethod )" -Body
```

```
$requestBody -Method Post -ContentType "application/json" -WebSession $session
    if ($response.StatusCode -eq 200)
    {
        $responseJson = ConvertFrom-Json $response
        if ($responseJson.id)
        {
            Write-Host $createSubmissionTemplateSuccessMessage
            $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$success
            $resultObj | Add-Member -MemberType NoteProperty -Name id -Value
$responseJson.id
        }
    }
    else
    {
        Write-Host $response.id
        Write-Host $createSubmissionTemplateFailMessage
        $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$failed
    }
}
catch [System.Net.WebException]
{
    Write-Host $createSubmissionTemplateFailMessage
    Write-Host "An exception was caught: $( $_.Exception.Message )"
    $resultObj | Add-Member -MemberType NoteProperty -Name status -Value $failed
}
return $resultObj
```

6.4 Create data set script

This sample is used for the creation of a data set entity. It includes another request that checks the data set creation status. The request that checks the status of creation is posted every 5 seconds until the creation is finished or more than 50 seconds have passed. When the script has finished successfully, the data set has been created.

```
#This script is used for the creation of a data set entity.
#It includes another request that checks the data set creation status.
#The request that checks the status of creation is posted every 5 seconds
#until the creation is finished or more than 50 seconds have passed.
#When the script has finished successfully, the data set has been created.

#specific input parameters for dataset creation
param (
    [parameter(Position = 1, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $dataSetName,
    [parameter(Position = 2, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $dataLot,
    [parameter(Position = 3, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $referenceDate,
    [parameter(Position = 4, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $workspaceId,
```

```
    [parameter(Position = 5, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $securityToken,
    [parameter(Position = 6, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $reportingBaseUri
)

#create the cookie using the security token received from input parameters, you
should not change it
$session = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$cookie = New-Object System.Net.Cookie
$cookie.Name = "__Secure-WKFS-Token"
$cookie.Value = "$securityToken"

$uriObject = [System.Uri]"$reportingBaseUri"
$cookie.Domain = $uriObject.Host
$cookie.Path = $uriObject.AbsolutePath

#add the cookie to the session object
$session.Cookies.Add($cookie);

$resultObj = New-Object -TypeName psobject

################## Data set creation ##################

#create the api url for dataset creation
#'rh-api' is common for all available functionalities, you should not change it
$baseApiUrl = $reportingBaseUri + '/rh-api/external/v1_0'
$APIMethod = "datasets/" + $workspaceId
$datasetIdUrl = 'datasets/getid/' + $workspaceId + '/' + $dataLot + '/' +
$referenceDate

#create the object(using the input parameters) needed for the request body and
convert it to json
$requestParameters = @{
    dataSetName = "$dataSetName"
    dataLot = "$dataLot"
    referenceDate = "$referenceDate"
}

$requestBody = $requestParameters | ConvertTo-Json
#call the rest endpoint for dataset creation and get the response

try
{
    Write-Host "Call the rest endpoint for dataset creation with parameters
dataSetName: " $dataSetName "  dataLot: " $dataLot " referenceDate: "
$referenceDate
    Invoke-RestMethod  "$( $baseApiUrl )/$( $APIMethod )" -Body $requestBody
-Method Post -ContentType "application/json" -WebSession $session | ConvertTo-Json
    Write-Host "Wait 30 seconds for dataset creation"
    start-sleep -seconds 30

    ################## Checking the creation status ##################

    $failed = "FAILED"
    $success = "SUCCESS"
    $maxIteration = 1
    $finished = $FALSE
```

```
    while ((-Not($finished)) -and ($maxIteration -lt 20))
    {
        Write-Host "Running try $( $maxIteration ) out of max 20 or until
successful"

        Write-Host "Call the rest endpoint for dataset id with URL parameters
workspaceId: " $workspaceId "  dataLot: " $dataLot " referenceDate: "
$referenceDate
        $datasetIdResponse = Invoke-WebRequest "$( $baseApiUrl )/$( $datasetIdUrl
)" -Method GET -WebSession $session

        if ($datasetIdResponse.StatusCode -eq 200)
        {
            Write-Host "Call the rest endpoint for dataset details with URL
parameters workspaceId: " $workspaceId "  datasetId: " $datasetIdResponse
            $datasetDetailsUrl = "datasets/" + $workspaceId + '/' +
$datasetIdResponse
            $datasetDetailsResponse = Invoke-WebRequest "$( $baseApiUrl )/$(
$datasetDetailsUrl )" -Method GET -WebSession $session

            Write-Host "Process the endpoint response to confirm the dataset
creation"
            if ($datasetDetailsResponse.StatusCode -eq 200)
            {
                $parsedCommandDetailsResponse = $datasetDetailsResponse.Content |
ConvertFrom-Json
                Write-Host "Endpoint response: "$parsedCommandDetailsResponse
                $parsedDatasetIdResponse = $datasetIdResponse.Content |
ConvertFrom-Json

                if($parsedCommandDetailsResponse.id -eq $parsedDatasetIdResponse)
                {
                    $resultObj | Add-Member -MemberType NoteProperty -Name id
-Value $parsedCommandDetailsResponse.id -Force
                    $resultObj | Add-Member -MemberType NoteProperty -Name status
-Value $success -Force
                    $finished = $TRUE
                    Write-Host "Dataset creation confirmed"
                }
            }
            else
            {
                $datasetDetailsResponse
                $resultObj | Add-Member -MemberType NoteProperty -Name status
-Value $failed -Force
                throw "Creation of data set failed!"
            }
        }

        $maxIteration = $maxIteration + 1
    }
}
catch [System.Net.WebException]
{
    $resultObj | Add-Member -MemberType NoteProperty -Name status -Value $failed
-Force
    Write-Host "Dataset creation failed"

    Write-Host "An exception was caught: $($_.Exception.Message)"
    $result = $_.Exception.Response.GetResponseStream()
    $reader = New-Object System.IO.StreamReader($result)
```

```
    $reader.BaseStream.Position = 0
    $reader.DiscardBufferedData()
    $responseBody = $reader.ReadToEnd();
    $_.Exception.Response
}

return $resultObj
```

6.5 Discard data set

This script is used for discarding a data set entity.

```
#This script is used for discarding a data set entity.

param (
    [parameter(Position = 1, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $workspaceId,
    [parameter(Position = 2, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $dataSetId,
    [parameter(Position = 3, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $deleteInputData,
    [parameter(Position = 4, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $forceDeleteResultData,
    [parameter(Position = 5, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $securityToken,
    [parameter(Position = 6, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $reportingBaseUri
)

#create the cookie using the security token received from input parameters, you
should not change it
$session = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$cookie = New-Object System.Net.Cookie
$cookie.Name = "__Secure-WKFS-Token"
$cookie.Value = "$securityToken"

$uriObject = [System.Uri]"$reportingBaseUri"
$cookie.Domain = $uriObject.Host
$cookie.Path = $uriObject.AbsolutePath

#add the cookie to the session object
$session.Cookies.Add($cookie);

$resultObj = New-Object -TypeName psobject
$failed = "FAILED"
$success = "SUCCESS"

$commandSuccessStatus = "COMPLETED"
```

```
$commandEndStatuses = $commandSuccessStatus, "STOPPED", "FAILED", "ABANDONED"
$commandStatus = "STARTING"
$maxIteration = 1

################## Discard data set ##################

#'rh-api' is common for all available functionalities, you should not change it
$baseApiUrl = $reportingBaseUri + '/rh-api/external/v1_0'
$APIMethod = "datasets/"+ $workspaceId +"/"+ $dataSetId

if (($deleteInputData -ne $null) -or ($forceDeleteResultData -ne $null))
{
    $APIMethod = $APIMethod + "?"
}

if ($deleteInputData -ne $null)
{
    $APIMethod = $APIMethod + "deleteInputData=" + $deleteInputData
}

if (($deleteInputData -ne $null) -and ($forceDeleteResultData -ne $null))
{
    $APIMethod = $APIMethod + "&forceDeleteResultData=" + $forceDeleteResultData
}
elseif (($deleteInputData -eq $null) -and ($forceDeleteResultData -ne $null))
{
    $APIMethod = $APIMethod + "forceDeleteResultData=" + $forceDeleteResultData
}

try
{
    Write-Host "Call rest end point $( $baseApiUrl )/$( $APIMethod ) to trigger
the discard of data set"
    $RawResponse = Invoke-RestMethod  "$( $baseApiUrl )/$( $APIMethod )" -Method
Delete -ContentType "application/json" -WebSession $session
    $Response = $RawResponse | ConvertTo-Json
    $Response | Out-String

    $baseApiUrl = $reportingBaseUri + '/rh-api/external/v1_0/processes'

    $ResponseObj = $Response | ConvertFrom-Json

    $APIMethod = "status?jobId=" + $ResponseObj.jobId

    while ((-Not($commandEndStatuses -contains $commandStatus)) -and
($maxIteration -lt 20))
    {
        $commandStatus = Invoke-WebRequest "$( $baseApiUrl )/$( $APIMethod )"
-Method GET -WebSession $session
        if ($commandStatus -like '*'+$commandSuccessStatus+'*')
        {
            $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$success
            Write-Host "Data set was discarded successfully!"
            break
        }
        if ($commandStatus -like '*FAILED*')
        {
            Write-Host "Discard of data set failed"
            break
        }
```

```
        $finished = $commandEndStatuses -contains $commandStatus

        if (-Not$finished)
        {
            Write-Host "Wait 5 seconds and try again.."
            start-sleep -seconds 5
        }
        $maxIteration = $maxIteration + 1
    }

    if (-Not($commandStatus -like '*'+$commandSuccessStatus+'*')){
        $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$failed
        Write-Host "Discard of data set failed!"
        throw "Discard of data set failed!"
    }
}
catch [System.Net.WebException]
{
     if ($($_.Exception.Message) -like "*The remote server returned an error:
(404) Not Found*"){    #means that the returned value is empty
        $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$success
        Write-Host "Could not find the data set with these parameters!"
     } else {
        $resultObj | Add-Member -MemberType NoteProperty -Name message -Value
$RawResponse
     }
}

return $resultObj

elseif (($deleteInputData -eq $null) -and ($forceDeleteResultData -ne $null))
{
    $APIMethod = $APIMethod + "forceDeleteResultData=" + $forceDeleteResultData
}

try
{
    Write-Host "Call rest end point $( $baseApiUrl )/$( $APIMethod ) to trigger
the discard of data set"
    $RawResponse = Invoke-RestMethod  "$( $baseApiUrl )/$( $APIMethod )" -Method
Delete -ContentType "application/json" -WebSession $session
    $Response = $RawResponse | ConvertTo-Json
    $Response | Out-String

    $baseApiUrl = $reportingBaseUri + '/rh-api/external/v1_0/processes'

    $ResponseObj = $Response | ConvertFrom-Json

    $APIMethod = "status?jobId=" + $ResponseObj.jobId

    while ((-Not($commandEndStatuses -contains $commandStatus)) -and
($maxIteration -lt 20))
    {
        $commandStatus = Invoke-WebRequest "$( $baseApiUrl )/$( $APIMethod )"
-Method GET -WebSession $session
        if ($commandStatus -like '*'+$commandSuccessStatus+'*')
        {
            $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$success
            Write-Host "Data set was discarded successfully!"
```

```
            break
        }
        if ($commandStatus -like '*FAILED*')
        {
            Write-Host "Discard of data set failed"
            break
        }

        $finished = $commandEndStatuses -contains $commandStatus

        if (-Not$finished)
        {
            Write-Host "Wait 5 seconds and try again.."
            start-sleep -seconds 5
        }
        $maxIteration = $maxIteration + 1
    }

    if (-Not($commandStatus -like '*'+$commandSuccessStatus+'*')){
        $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$failed
        Write-Host "Discard of data set failed!"
        throw "Discard of data set failed!"
    }
}
catch [System.Net.WebException]
{
     if ($($_.Exception.Message) -like "*The remote server returned an error:
(404) Not Found*"){    #means that the returned value is empty
        $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$success
        Write-Host "Could not find the data set with these parameters!"
     } else {
        $resultObj | Add-Member -MemberType NoteProperty -Name message -Value
$RawResponse
     }
}

return $resultObj
```

6.6 Lookup data set ID

This sample is used for the lookup of a data set identifier by data lot, reference day, and workspace ID. When the script has finished successfully, the data set ID is returned.

```
#Script does a lookup of a data set id. Data set is identified using the
parameters: workspaceId, dataLot and referenceDate

#specific input parameters for lookup dataset
param (
    [parameter(Position = 1, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $dataLot,
    [parameter(Position = 2, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $referenceDate,
    [parameter(Position = 3, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
```

```
    [ValidateNotNullOrEmpty()]
    $workspaceId,
    [parameter(Position = 4, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $securityToken,
    [parameter(Position = 5, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $reportingBaseUri
)
[System.Net.ServicePointManager]::ServerCertificateValidationCallback = { $true }
#create the cookie using the security token received from input parameters, you
should not change it
$session = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$cookie = New-Object System.Net.Cookie
$cookie.Name = "__Secure-WKFS-Token"
$cookie.Value = "$securityToken"

$uriObject = [System.Uri]"$reportingBaseUri"
$cookie.Domain = $uriObject.Host
$cookie.Path = $uriObject.AbsolutePath

#add the cookie to the session object
$session.Cookies.Add($cookie);

$resultObj = New-Object -TypeName psobject
$failed = "FAILED"
$success = "SUCCESS"

################## Data set lookup ##################

$baseApiUrl = $reportingBaseUri + '/rh-api/external/v1_0'
$APIMethod = "datasets/getid/" + $workspaceId + "/" + $dataLot + "/"
+$referenceDate

try
{
    Write-Host "Call rest endpoint $( $baseApiUrl )/$( $APIMethod ) to get the
data set id"
    $Response = Invoke-RestMethod  "$( $baseApiUrl )/$( $APIMethod )" -Method Get
-ContentType "application/json" -WebSession $session | ConvertTo-Json
    Write-Host "Response received: "$Response
    if ($Response )
    {
        Write-Host "Lookup was successful"
        $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$success
        $resultObj | Add-Member -MemberType NoteProperty -Name id -Value $Response
    }
    else
    {
        $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$failed
        throw "Lookup of dataset failed!"
    }
}

catch [System.Net.WebException]
{
    Write-Host "An exception was caught: $($_.Exception.Message)"
    $resultObj | Add-Member -MemberType NoteProperty -Name status -Value $failed
```

```
    throw "Lookup of dataset failed!"
}

return $resultObj
```

6.7 Validate input data

This sample is used to validate input data and includes another request which checks the validation status. The request that checks the status of validation is posted every 5 seconds until the validation is finished or more than 50 seconds have passed. When the script has finished, the validation status is returned.

```
#Script triggers a data set input data validation and checks if the validation is
successful or fails

#specific input parameters for validate input data
param (
    [parameter(Position = 1, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $dataSetId,
    [parameter(Position = 2, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $workspaceId,
    [parameter(Position = 3, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $securityToken,
    [parameter(Position = 4, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $reportingBaseUri
)

#create the cookie using the security token received from input parameters, you
should not change it
$session = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$cookie = New-Object System.Net.Cookie
$cookie.Name = "__Secure-WKFS-Token"
$cookie.Value = "$securityToken"

$uriObject = [System.Uri]"$reportingBaseUri"
$cookie.Domain = $uriObject.Host
$cookie.Path = $uriObject.AbsolutePath

#add the cookie to the session object
$session.Cookies.Add($cookie);

################## Validate input data ##################

#create the api url for validation
#'rh-api' is common for all available functionalities, you should not change it
$baseApiUrl = $reportingBaseUri + "/rh-api/external/v1_0"
$APIMethod = "datasets/" + $workspaceId + "/" + $dataSetId + "/input-data/
validation"

$resultObj = New-Object -TypeName psobject
#call the rest endpoint for validation input data and get the response

try
```

```
{
    Write-Host "Call POST rest endpoint at: $( $baseApiUrl )/$( $APIMethod )"
    $Response = Invoke-RestMethod  "$( $baseApiUrl )/$( $APIMethod )" -Method Post
-ContentType "application/json" -WebSession $session | ConvertTo-Json

    ################## Checking the validation status ##################

    $commandSuccessStatus = "VALID"
    $commandEndStatuses = $commandSuccessStatus, "INVALID"
    $commandStatus = "IN_PROGRESS"
    $resultObj = New-Object -TypeName psobject
    #call the rest endpoint for validation input data and get the response
try
{
    Write-Host "Call POST rest endpoint at: $( $baseApiUrl )/$( $APIMethod )"
    $Response = Invoke-RestMethod  "$( $baseApiUrl )/$( $APIMethod )" -Method Post
-ContentType "application/json" -WebSession $session | ConvertTo-Json

################## Checking the validation status ##################

    $commandSuccessStatus = "VALID"
    $commandEndStatuses = $commandSuccessStatus, "INVALID"
    $commandStatus = "IN_PROGRESS"
    $failed = "FAILED"
    $success = "SUCCESS"
    $APIMethod = $APIMethod + "/status"
    $maxIteration = 1
    while ((-Not($commandEndStatuses -contains $commandStatus)) -and
($maxIteration -lt 20))
    {
        Write-Host "Running try $( $maxIteration ) out of max 20 or until
successful"
        Write-Host "Call rest endpoint $( $baseApiUrl )/$( $APIMethod ) to check
validation status"
        $commandDetailsResponse = Invoke-WebRequest "$( $baseApiUrl )/$(
$APIMethod )" -Method GET -WebSession $session

        if ($commandDetailsResponse.StatusCode -eq 200)
        {
            $parsedCommandDetailsReponse = $commandDetailsResponse.Content |
ConvertFrom-Json

            $commandStatus = $parsedCommandDetailsReponse.status
            Write-Host "Validation status is: "$commandStatus
            if ($commandStatus -eq "VALID")
            {
                Write-Host "Validation completed successfully"
                $resultObj | Add-Member -MemberType NoteProperty -Name status
-Value $success
                $resultObj | Add-Member -MemberType NoteProperty -Name id -Value
                $Response
                break
            }
        }
        else
        {
            $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$failed
            throw "Validate input data failed!"
        }

        $finished = $commandEndStatuses -contains $commandStatus
```

```
        if (-Not$finished)
        {
            Write-Host "As the validation status is not in a final state we will
wait 5 seconds and try again"
            start-sleep -seconds 5
        }
        $maxIteration = $maxIteration + 1
    }

    if ($commandStatus -eq "INVALID")
    {
        $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$failed
        throw "Validate input data failed!"
    }
}
catch [System.Net.WebException]
{
    Write-Host "An exception was caught: $($_.Exception.Message)"
    $resultObj | Add-Member -MemberType NoteProperty -Name status -Value $failed

    Write-Host "An exception was caught: $($_.Exception.Message)"
    $result = $_.Exception.Response.GetResponseStream()
    $reader = New-Object System.IO.StreamReader($result)
    $reader.BaseStream.Position = 0
    $reader.DiscardBufferedData()
    $responseBody = $reader.ReadToEnd();
    $_.Exception.Response
    throw "Validate input data failed!"
}

return $resultObj
```

6.8 Enrich input data

This sample is used to enrich input data and includes another request that checks the enrichment status. The request that checks the status of enrichment is posted every 5 seconds until the enrichment process has finished, or more than 50 seconds have passed. When the script has finished, the status of enrichment is returned.

```
#Script retrieves the id of a process parameter set for given workspace and
enrichmentId, triggers the enrichment
#of a data set using the id of the process parameter set previously received and
checks if the process is successful or fails

#specific input parameters for enrichment input data
param (
    [parameter(Position = 1, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $dataSetId,
    [parameter(Position = 2, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $enrichmentId,
    [parameter(Position = 3, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $workspaceId,
```

```
    [parameter(Position = 4, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $securityToken,
    [parameter(Position = 5, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $reportingBaseUri
)

#create the cookie using the security token received from input parameters, you
should not change it
$session = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$cookie = New-Object System.Net.Cookie
$cookie.Name = "__Secure-WKFS-Token"
$cookie.Value = "$securityToken"

$uriObject = [System.Uri]"$reportingBaseUri"
$cookie.Domain = $uriObject.Host
$cookie.Path = $uriObject.AbsolutePath

#add the cookie to the session object
$session.Cookies.Add($cookie);
$resultObj = New-Object -TypeName psobject

################## Enrichment input data ##################

$baseApiUrl_process = $reportingBaseUri + '/rh-api/external/v1_0'
$APIMethod_process = "processing-params/" + $workspaceId + "?enrichmentProcessId="
+ $enrichmentId

#create the api url for enrichment
#'rh-api' is common for all available functionalities, you should not change it
$baseApiUrl = $reportingBaseUri + '/rh-api/external/v1_0'
$APIMethod = 'datasets/'+ $workspaceId +'/'+ $dataSetId +'/enrichment'

try
{
    Write-Host "Call rest endpoint $( $baseApiUrl_process )/$( $APIMethod_process
) to get the process parameters set id"
    $Response_process = Invoke-RestMethod  "$( $baseApiUrl_process )/$(
$APIMethod_process )" -Method Get -ContentType "application/json" -WebSession
$session | ConvertTo-Json
    Write-Host "The call response is:" $Response_process

    #create the object(using the input parameters) needed for the request body and
convert it to json
    $requestParameters = @{
        enrichmentId = $Response_process
    }

    $requestBody = $requestParameters | ConvertTo-Json

Write-Host "Call rest endpoint $( $baseApiUrl )/$( $APIMethod ) to trigger the
enrichment process using the process parameter id previously obtained"
    $Response = Invoke-RestMethod  "$( $baseApiUrl )/$( $APIMethod )" -Body
$requestBody -Method Post -ContentType "application/json" -WebSession $session |
ConvertTo-Json
    ################## Checking the validation status ##################

    $commandSuccessStatus = "COMPLETED"
    $commandEndStatuses = $commandSuccessStatus, "FAILED", "ENRICHED"
```

```
    $commandStatus = "STARTING"
    $failed = "FAILED"
    $success = "SUCCESS"

    $APIMethod = "processes/status?jobId=" + $Response
    $maxIteration = 1
    while ((-Not($commandEndStatuses -contains $commandStatus)) -and
($maxIteration -lt 20))
    {
        Write-Host "Running try $( $maxIteration ) out of max 20 or until
successful"
        Write-Host "Call rest end point: $( $baseApiUrl )/$( $APIMethod ) to
retrieve the enrichment process status"
        $commandDetailsResponse = Invoke-WebRequest "$( $baseApiUrl )/$(
$APIMethod )" -Method GET -WebSession $session

        if ($commandDetailsResponse.StatusCode -eq 200)
        {
            $parsedCommandDetailsReponse = $commandDetailsResponse.Content

            $commandStatus = $parsedCommandDetailsReponse

            if ($commandStatus -eq "COMPLETED" -Or $commandStatus -eq "ENRICHED")
            {
                Write-Host "Enrichment process was successful"
                $resultObj | Add-Member -MemberType NoteProperty -Name status
-Value $success
                break
            }
        }
        else
        {
            Write-Host "Rest call failed"
            $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$failed
        }

        $finished = $commandEndStatuses -contains $commandStatus

        if (-Not$finished)
        {
            Write-Host "As the process status is not in a final state we will wait
5 seconds and try again"
            start-sleep -seconds 5
        }
        $maxIteration = $maxIteration + 1
    }

    if ($commandStatus -eq $failed)
    {
        Write-Host "Enrichment process failed"
        $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$failed
    }
}
catch [System.Net.WebException]
{
    Write-Host "An exception was caught: $($_.Exception.Message)"
    $resultObj | Add-Member -MemberType NoteProperty -Name status -Value $failed
}

return $resultObj
```

6.9 Lookup created result sets

This sample script retrieves result set IDs and names created from the input data of a data set of type DF_lot based, via an enrichment process.

```
#Script does a lookup of the result sets of a data set. Data set is identified
using the parameters: workspaceId and dataSetId

#specific input parameters for lookup result sets
param (
    [parameter(Position = 1, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $dataSetId,
    [parameter(Position = 2, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $workspaceId,
    [parameter(Position = 3, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $securityToken,
    [parameter(Position = 4, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $reportingBaseUri
)

#create the cookie using the security token received from input parameters, you
should not change it
$session = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$cookie = New-Object System.Net.Cookie
$cookie.Name = "__Secure-WKFS-Token"
$cookie.Value = "$securityToken"

$uriObject = [System.Uri]"$reportingBaseUri"
$cookie.Domain = $uriObject.Host
$cookie.Path = $uriObject.AbsolutePath

#add the cookie to the session object
$session.Cookies.Add($cookie);

$resultObj = New-Object -TypeName psobject
$failed = "FAILED"
$success = "SUCCESS"

################## Result sets lookup ##################

#'rh-api' is common for all available functionalities, you should not change it
$baseApiUrl = $reportingBaseUri + '/rh-api/external/v1_0'
$APIMethod = "datasets/" + $workspaceId + "/" + $dataSetId +"/resultsets"

try
{
    Write-Host "Call rest endpoint $( $baseApiUrl )/$( $APIMethod ) to get the
data set id"
    $Response = Invoke-RestMethod  "$( $baseApiUrl )/$( $APIMethod )" -Method Get
-ContentType "application/json" -WebSession $session | ConvertTo-Json
    Write-Host "Response received: "$Response
  if ($Response)
    {
```

```
        $parsed = $Response | ConvertFrom-Json
        $myArray = @()

        foreach ($i in $parsed.content)
        {
            $myObject = New-Object System.Object

            $myObject | Add-Member -type NoteProperty -name id -Value $i.id
            $myObject | Add-Member -type NoteProperty -name name -Value
$i.dataSetName

            $myArray += $myObject
            Write-Host "One value found with id: $($i.id) and name:
$($i.dataSetName)"
        }
        Write-Host "Lookup was successful"
        $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$success
        $resultObj | Add-Member -MemberType NoteProperty -Name id -Value $myArray
    }
    else
    {
        $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$failed
        throw "Lookup failed!"
    }
}
catch [System.Net.WebException]
{
    Write-Host "An exception was caught: $($_.Exception.Message)"
    $resultObj | Add-Member -MemberType NoteProperty -Name status -Value $failed
}

return $resultObj
```

6.10 Add submission

This sample is used for the creation of a submission entity and includes another request which checks the submission creation status. The request that checks the status of creation is posted every 5 seconds until the creation process is finished, or more than 50 seconds have passed. When the script has finished successfully, the submission has been created.

```
#This sample is used for the creation of a submission entity
#and includes another request which checks the submission creation status.
#The request that checks the status of creation is posted every 5 seconds
#until the creation is finished or more than 50 seconds have passed.
#When the script has finished successfully, the submission has been created.
#specific input parameters for submission creation
param (
    [parameter(Position = 1, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $templateId,
    [parameter(Position = 2, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $submissionName,
    [parameter(Position = 3, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $reportingDeadline,
    [parameter(Position = 4, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
```

```
    $submissionReferenceDate,
    [parameter(Position = 5, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $submitBy,
    [parameter(Position = 6, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $workspaceId,
    [parameter(Position = 7, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $securityToken,
    [parameter(Position = 8, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $reportingBaseUri
)

#create the cookie using the security token received from input parameters, you
should not change it
$session = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$cookie = New-Object System.Net.Cookie
$cookie.Name = "__Secure-WKFS-Token"
$cookie.Value = "$securityToken"
$failed = "FAILED"
$success = "SUCCESS"

$uriObject = [System.Uri]"$reportingBaseUri"
$cookie.Domain = $uriObject.Host
$cookie.Path = $uriObject.AbsolutePath

#add the cookie to the session object
$session.Cookies.Add($cookie);

################## Submission creation #################

#create the api url for submission creation
#'rh-api' is common for all available functionalities, you should not change it
$baseApiUrl = $reportingBaseUri + '/rh-api/external/v1_0/submissions'
$APIMethod = $workspaceId

#create the object(using the input parameters) needed for the request body and
convert it to json
$requestParameters = @{
    submissionTemplateId = "$templateId"
    submissionName = "$submissionName"
    referenceDate = "$submissionReferenceDate "
    reportingDeadline = "$reportingDeadline"
    submitBy = "$submitBy"
}
$requestBody = $requestParameters | ConvertTo-Json
#call the rest endpoint for submission creation and get the response
$resultObj = New-Object -TypeName psobject

$commandSuccessStatus = "COMPLETED"
$commandEndStatuses = $commandSuccessStatus, "STOPPED", "FAILED", "ABANDONED"
$commandStatus = "STARTING"
$maxIteration = 1

try
{
    Write-Host "Call the rest endpoint for submission creation with parameters:
submissionTemplateId:" $templateId " submissionName:" $submissionName
"referenceDate: " $submissionReferenceDate "reportingDeadline: "
$reportingDeadline "submitBy: " $submitBy
```

```
    $Response = Invoke-RestMethod  "$( $baseApiUrl )/$( $APIMethod )" -Body
$requestBody -Method Post -ContentType "application/json" -WebSession $session |
ConvertTo-Json

    ################# Checking the creation status #################

    $baseApiUrl = $reportingBaseUri + '/rh-api/external/v1_0/processes'

    $APIMethod = "status?jobId=" + $Response

    while ((-Not($commandEndStatuses -contains $commandStatus)) -and
($maxIteration -lt 20))
    {
        $commandStatus = Invoke-WebRequest "$( $baseApiUrl )/$( $APIMethod )"
-Method GET -WebSession $session
        if ($commandStatus -like '*'+$commandSuccessStatus+'*')
        {
            $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$success
            Write-Host "Creation of submission was successful"
            break
        }
        if ($commandStatus -like '*FAILED*')
        {
            Write-Host "Creation of submission failed"
            break
        }

        $finished = $commandEndStatuses -contains $commandStatus

        if (-Not$finished)
        {
            Write-Host "Wait 5 seconds and try again.."
            start-sleep -seconds 5
        }
        $maxIteration = $maxIteration + 1
    }

    if (-Not($commandStatus -like '*'+$commandSuccessStatus+'*')){
        $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$failed
        Write-Host "Creation of submission failed"
        throw "Creation of submission failed!"
    }
}
catch [System.Net.WebException]
{
    Write-Host "An exception was caught: $($_.Exception.Message)"
    $resultObj | Add-Member -MemberType NoteProperty -Name status -Value $failed
    Write-Host "Creation of submission failed"
    throw "Creation of submission failed!"
}

return $resultObj
```

6.11 Lookup submission ID

This sample is used for lookup of a submission identifier by report set, reporting date, reporting entity, workspace ID and submission name. When the script has finished successfully, the submission ID is returned.

```
#Script does a lookup of a submission status. Submission is identified using the
parameters: workspaceId, reportingEntityId, reportingDate and reportSetId

#specific input parameters for lookup submission
param (
    [parameter(Position = 1, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $reportSetId,
    [parameter(Position = 2, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $reportingDate,
    [parameter(Position = 3, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $reportingEntity,
    [parameter(Position = 4, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $workspaceId,
    [parameter(Position = 5, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()],
    $submissionName,
    [parameter(Position = 6, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $securityToken,
    [parameter(Position = 7, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $reportingBaseUri
)

#create the cookie using the security token received from input parameters, you
should not change it
$session = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$cookie = New-Object System.Net.Cookie
$cookie.Name = "__Secure-WKFS-Token"
$cookie.Value = "$securityToken"

$uriObject = [System.Uri]"$reportingBaseUri"
$cookie.Domain = $uriObject.Host
$cookie.Path = $uriObject.AbsolutePath

#add the cookie to the session object
$session.Cookies.Add($cookie);

$resultObj = New-Object -TypeName psobject
$failed = "FAILED"
$success = "SUCCESS"

################## Submission lookup ##################

$baseApiUrl = $reportingBaseUri + '/rh-api/external/v1_0/submissions'
$APIMethod = $workspaceId + '/' + $reportingEntity + "/" + $reportingDate + "/" +
$reportSetId + "/" + $submissionName
$resultObj = New-Object -TypeName psobject
try
{
    Write-Host "Call rest endpoint $( $baseApiUrl )/$( $APIMethod ) to check
```

```
submission status"
    $Response = Invoke-RestMethod  "$( $baseApiUrl )/$( $APIMethod )" -Method Get
-ContentType "application/json" -WebSession $session | ConvertTo-Json
    Write-Host "Response received: "$Response
     if ($Response)
    {
        Write-Host "Lookup was successful"
        $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$success

        $parsedCommandDetailsResponse = $Response | ConvertFrom-Json

        $resultObj | Add-Member -MemberType NoteProperty -Name id -Value
$parsedCommandDetailsResponse.id
        $resultObj | Add-Member -MemberType NoteProperty -Name subStatus -Value
$parsedCommandDetailsResponse.submissionStatus
    }
    else
    {
        $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$failed
        throw "Lookup of submission failed!"
    }
}
catch [System.Net.WebException]
{
    Write-Host "An exception was caught: $( $_.Exception.Message )"
    $resultObj | Add-Member -MemberType NoteProperty -Name status -Value $failed
    throw "Lookup of submission failed!"
}

return $resultObj
```

6.12 Add data set to submission

This sample is used to assign a result set to a submission entity which has already been created. This script also includes another request, which is posted after 3 seconds and returns the status of previous operation. When the script has finished successfully, the result set is assigned.

```
#This script is used to assign a result set to a submission entity which has
already been created
#Also includes another request, which is posted after 3 seconds and returns the
status of previous operation.
#When the script has finished successfully, the result set is assigned.

param (
    [parameter(Position = 1, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $resultSetIds,
    [parameter(Position = 2, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $selectAll,
    [parameter(Position = 3, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $submissionId,
    [parameter(Position = 4, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
```

```
    [ValidateNotNullOrEmpty()]
    $workspaceId,
    [parameter(Position = 5, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $securityToken,
    [parameter(Position = 6, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $reportingBaseUri
)

#create the cookie using the security token received from input parameters, you
should not change it
$session = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$cookie = New-Object System.Net.Cookie
$cookie.Name = "__Secure-WKFS-Token"
$cookie.Value = "$securityToken"

$uriObject = [System.Uri]"$reportingBaseUri"
$cookie.Domain = $uriObject.Host
$cookie.Path = $uriObject.AbsolutePath

#add the cookie to the session object
$session.Cookies.Add($cookie);

$resultObj = New-Object -TypeName psobject

################## Add data set to submission ##################

$baseApiUrl = $reportingBaseUri + '/rh-api/external/v1_0'
$APIMethod = "submissions/" + $workspaceId + "/" + $submissionId + "/resultsets"

#create the object(using the input parameters) needed for the request body and
convert it to json
$requestParameters = @{
    resultSetIds = $resultSetIds -split ','
    selectAll = "$selectAll"
}

$requestBody = $requestParameters | ConvertTo-Json
#call the rest endpoint for dataset creation and get the response
try
{
    Write-Host "Call the rest endpoint for adding result set to submission"
    Invoke-RestMethod  "$( $baseApiUrl )/$( $APIMethod )" -Body $requestBody
-Method Post -ContentType "application/json" -WebSession $session | ConvertTo-Json

    ################## Checking the status ##################
    Write-Host "Wait 3 seconds.."
    start-sleep -seconds 3

    $failed = "FAILED"
    $success = "SUCCESS"
    $APIMethod = $APIMethod + "/count"
    $commandDetailsResponse = Invoke-WebRequest "$( $baseApiUrl )/$( $APIMethod )"
-Method GET -WebSession $session

    $parsedCommandDetailsReponse = $commandDetailsResponse | ConvertFrom-Json

    if ($parsedCommandDetailsReponse.count -gt 0)
    {
```

```
        $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$success
        Write-Host "Result set assigned to submission successfully"
    }
    else
    {
        $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$failed
        Write-Host "Failed to assign result set to submission"
    }
}
catch [System.Net.WebException]
{
    Write-Host "Failed to assign result set to submission"
    Write-Host "An exception was caught: $($_.Exception.Message)"
    $resultObj | Add-Member -MemberType NoteProperty -Name status -Value $failed
}

return $resultObj
```

6.13 Discard submission

This sample is used for the deletion of a submission entity and includes another request that checks the submission deletion status. The request that checks the status of deletion is posted every 5 seconds until the deletion is finished, or more than 100 seconds have passed. When the script has finished successfully, the submission has been deleted.

```
#This sample is used for the deletion of a submission entity
#and includes another request which checks the submission deletion status.
#The request that checks the status of deletion is posted every 5 seconds
#until the deletion is finished or more than 100 seconds have passed.
#When the script has finished successfully, the submission has been deleted.

#specific input parameters for submission creation
param (
    [parameter(Position = 1, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $workspaceId,
    [parameter(Position = 6, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $submissionId,
    [parameter(Position = 7, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $securityToken,
    [parameter(Position = 8, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $reportingBaseUri
)

#create the cookie using the security token received from input parameters, you
should not change it
$session = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$cookie = New-Object System.Net.Cookie
$cookie.Name = "__Secure-WKFS-Token"
$cookie.Value = "$securityToken"
$failed = "FAILED"
$success = "SUCCESS"

$uriObject = [System.Uri]"$reportingBaseUri"
$cookie.Domain = $uriObject.Host
```

```
$cookie.Path = $uriObject.AbsolutePath

#add the cookie to the session object
$session.Cookies.Add($cookie);

################## Submission deletion ##################

#create the api url for submission deletion
#'rh-api' is common for all available functionalities, you should not change it
$baseApiUrl = $reportingBaseUri + '/rh-api/external/v1_0/submissions'
$APIMethod = $workspaceId + '/' + $submissionId

#call the rest endpoint for submission deletion and get the response
$resultObj = New-Object -TypeName psobject

$commandSuccessStatus = "COMPLETED"
$commandEndStatuses = $commandSuccessStatus, "STOPPED", "FAILED", "ABANDONED"
$commandStatus = "STARTING"
$maxIteration = 1

try
{
    Write-Host "Call the rest endpoint for submission deletion with parameters:
workspaceId:" $workspaceId " submissionId:" $submissionId
    $Response = Invoke-RestMethod  "$( $baseApiUrl )/$( $APIMethod )" -Method
Delete -ContentType "application/json" -WebSession $session | ConvertTo-Json

    ###################### Checking the deletion status ##################

    $baseApiUrl = $reportingBaseUri + '/rh-api/external/v1_0/processes'

    $APIMethod = "status?jobId=" + $Response

    Write-Host "Deletion jobId is: " $Response
    while ((-Not($commandEndStatuses -contains $commandStatus)) -and
($maxIteration -lt 20))
    {
         Write-Host "Getting job status from: $( $baseApiUrl )/$( $APIMethod )"
        $commandStatus = Invoke-WebRequest "$( $baseApiUrl )/$( $APIMethod )"
-Method GET -WebSession $session
        Write-Host "Job status is: " $commandStatus
        if ($commandStatus -like '*'+$commandSuccessStatus+'*')
        {
            $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$success
            Write-Host "Deletion of submission was successful"
            break
        }
        if ($commandStatus -like '*FAILED*')
        {
            Write-Host "Deletion of submission failed"
            break
        }

        $finished = $commandEndStatuses -contains $commandStatus

        if (-Not$finished)
        {
            Write-Host "Wait 5 seconds and try again.."
            start-sleep -seconds 5
        }
        $maxIteration = $maxIteration + 1
```

```
    }

    if (-Not($commandStatus -like '*'+$commandSuccessStatus+'*')){
        $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$failed
        Write-Host "Deletion of submission failed"
        throw "Deletion of submission failed!"
    }
}
catch [System.Net.WebException]
{
    Write-Host "An exception was caught: $($_.Exception.Message)"
    $resultObj | Add-Member -MemberType NoteProperty -Name status -Value $failed
    Write-Host "Deletion of submission failed"
    throw "Deletion of submission failed!"
}

return $resultObj
```

6.14 Submission references

6.14.1 Add submission references to submission

```
#This script is used to add submission references
param (
    [parameter(Position = 1, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $submissionInformationId,
    [parameter(Position = 2, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $reportingSetId,
    [parameter(Position = 3, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $reportingSetReferenceName,
    [parameter(Position = 4, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $submissionId,
    [parameter(Position = 5, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $workspaceId,
    [parameter(Position = 6, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $securityToken,
    [parameter(Position = 7, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $reportingBaseUri
)

#create the cookie using the security token received from input parameters, you
should not change it
$session = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$cookie = New-Object System.Net.Cookie
```

```
$cookie.Name = "__Secure-WKFS-Token"
$cookie.Value = "$securityToken"

$uriObject = [System.Uri]"$reportingBaseUri"
$cookie.Domain = $uriObject.Host
$cookie.Path = $uriObject.AbsolutePath

#add the cookie to the session object
$session.Cookies.Add($cookie);

$resultObj = New-Object -TypeName psobject
################## Adding reference to submission #################

#'rh-api' is common for all available functionalities, you should not change it
$baseApiUrl = $reportingBaseUri + '/rh-api/external/v1_0/submissions'
$APIMethod = $workspaceId + '/' + $submissionId + '/references'

#create the object(using the input parameters) needed for the request body and
convert it to json
$requestParameters = @{
    submissionInformationId = "$submissionInformationId"
    reportingSetId = "$reportingSetId"
    reportingSetReferenceName = "$reportingSetReferenceName"
}

$requestBody = $requestParameters | ConvertTo-Json

#call the rest endpoint and get the response
$resultObj = New-Object -TypeName psobject

try
{
    Write-Host "Call the rest endpoint for adding submission reference"
    $Response = Invoke-RestMethod  "$( $baseApiUrl )/$( $APIMethod )" -Body
$requestBody -Method Post -ContentType "application/json" -WebSession $session |
ConvertTo-Json
}
catch [System.Net.WebException]
{
    Write-Host "An exception was caught: $( $_.Exception.Message )"
    $resultObj | Add-Member -MemberType NoteProperty -Name status -Value $failed
    Write-Host "Adding reference failed"
    throw "Adding reference failed!"
}

return $resultObj
```

6.14.2 Remove submission references from submission

```
#This script is used to remove submission references
param (
    [parameter(Position = 1, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $submissionReferenceId,
    [parameter(Position = 2, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $submissionId,
    [parameter(Position = 3, Mandatory = $true, ValueFromPipeline = $true,
```

```
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $workspaceId,
    [parameter(Position = 4, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $securityToken,
    [parameter(Position = 5, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $reportingBaseUri
)

#create the cookie using the security token received from input parameters, you
should not change it
$session = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$cookie = New-Object System.Net.Cookie
$cookie.Name = "__Secure-WKFS-Token"
$cookie.Value = "$securityToken"

$uriObject = [System.Uri]"$reportingBaseUri"
$cookie.Domain = $uriObject.Host
$cookie.Path = $uriObject.AbsolutePath

#add the cookie to the session object
$session.Cookies.Add($cookie);

$resultObj = New-Object -TypeName psobject

################## Removing reference from submission ##################

#'rh-api' is common for all available functionalities, you should not change it
$baseApiUrl = $reportingBaseUri + '/rh-api/external/v1_0/submissions'
$APIMethod = $workspaceId + '/' + $submissionId + '/references' + '/' +
$submissionReferenceId

#call the rest endpoint and get the response
$resultObj = New-Object -TypeName psobject

try
{
    Write-Host "Call the rest endpoint for removing submission reference"
    $Response = Invoke-RestMethod  "$( $baseApiUrl )/$( $APIMethod )" -Method
Delete -ContentType "application/json" -WebSession $session | ConvertTo-Json
}
catch [System.Net.WebException]
{
    Write-Host "An exception was caught: $( $_.Exception.Message )"
    $resultObj | Add-Member -MemberType NoteProperty -Name status -Value $failed
    Write-Host "Removing reference failed"
    throw "Removing reference failed!"
}

return $resultObj
```

6.15 Post process

6.15.1 Lookup after post process

This sample is used to find all post processes. All post process IDs are returned after the script has finished

successfully.

```
#Script does lookup of post processes for submission

param (
    [parameter(Position = 1, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $submissionId,
    [parameter(Position = 2, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $workspaceId,
    [parameter(Position = 3, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $securityToken,
    [parameter(Position = 4, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $reportingBaseUri
)

#create the cookie using the security token received from input parameters, you
should not change it
$session = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$cookie = New-Object System.Net.Cookie
$cookie.Name = "__Secure-WKFS-Token"
$cookie.Value = "$securityToken"

$uriObject = [System.Uri]"$reportingBaseUri"
$cookie.Domain = $uriObject.Host
$cookie.Path = $uriObject.AbsolutePath

#add the cookie to the session object
$session.Cookies.Add($cookie);

$resultObj = New-Object -TypeName psobject
$failed = "FAILED"
$success = "SUCCESS"

################## Result set post process lookup ##################

#'rh-api' is common for all available functionalities, you should not change it
$baseApiUrl = $reportingBaseUri + '/rh-api/external/v1_0'
$APIMethod = "submissions/"+ $workspaceId +"/"+ $submissionId+"/post-processing/
links"
try
{
    Write-Host "Call rest end point $( $baseApiUrl )/$( $APIMethod )"
    $Response = Invoke-RestMethod  "$( $baseApiUrl )/$( $APIMethod )" -Method Get
-ContentType "application/json" -WebSession $session | ConvertTo-Json
    Write-Host "We received response: "$Response
    if ($Response)
    {
        $parsed = $Response | ConvertFrom-Json

        $myArray = @()

        foreach ($i in $parsed.value)
        {
Write-Host "One value: $($i)"
```

```
            $myArray += $i
        }

        $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$success
        $resultObj | Add-Member -MemberType NoteProperty -Name id -Value $myArray
        Write-Host "Script run was successful"
    }
    else
    {
        Write-Host "Script run failed"
        $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$failed
    }
}
catch [System.Net.WebException]
{
    if ($($_.Exception.Message) -like "*The remote server returned an error: (404)
Not Found*"){    #means that the returned value is empty
       $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$success
    } else {
       $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$failed
    }
    Write-Host "Script run failed"
Write-Host $($_.Exception)
    Write-Host "An exception was caught: $($_.Exception.Message)"
}

return $resultObj
```

6.15.2 Call the post processes

This sample script is used to call all post processes that were found and to check the status of postprocessing. The request that checks the status of postprocessing is posted every 5 seconds until the postprocessing has finished, or more than 50 seconds have passed.

When the script has finished, all post processes are executed.

```
#Script triggers a submission post process and checks if the process is successful
or fails

#specific input parameters for post procession
param (
    [parameter(Position = 1, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $submissionId,
    [parameter(Position = 2, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $workspaceId,
    [parameter(Position = 3, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $reportSetPostProcessingId,
    [parameter(Position = 4, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $securityToken,
    [parameter(Position = 5, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $reportingBaseUri
```

```
)

#create the cookie using the security token received from input parameters, you
should not change it
Write-Host "Creating session object for authenticated requests"
$session = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$cookie = New-Object System.Net.Cookie
$cookie.Name = "__Secure-WKFS-Token"
$cookie.Value = "$securityToken"
$failed = "FAILED"
$success = "SUCCESS"

$uriObject = [System.Uri]"$reportingBaseUri"
$cookie.Domain = $uriObject.Host
$cookie.Path = $uriObject.AbsolutePath

#add the cookie to the session object
$session.Cookies.Add($cookie);

################## Post procession ##################

#create the api url for dataset creation
#'rh-api' is common for all available functionalities, you should not change it

$baseApiUrl_id = $reportingBaseUri + '/rh-api/external/v1_0/submissions/'
#$APIMethod_id = "report-sets/getByIds?reportSetId=" + $resultSetIds

$baseApiUrl = $reportingBaseUri + '/rh-api/external/v1_0/submissions/' +
$workspaceId + '/' + $submissionId

#call the rest endpoint for dataset creation and get the response
$resultObj = New-Object -TypeName psobject

$commandSuccessStatus = "COMPLETED"
$commandEndStatuses = $commandSuccessStatus
$commandStatus = "STARTING"
$maxIteration = 1

try
{
    $APIMethod = "post-
processing?reportSetPostProcessingId="+$reportSetPostProcessingId

    Write-Host "Do a POST call the rest endpoint: $( $baseApiUrl )/$( $APIMethod
)"
    $Response = Invoke-RestMethod  "$( $baseApiUrl )/$( $APIMethod )" -Method Post
-ContentType "application/json" -WebSession $session | ConvertTo-Json

    Write-Host "We receive the response with the jobId: "$Response

    ################## Checking the procession status ##################

    $baseApiUrl = $reportingBaseUri + '/rh-api/external/v1_0/processes'

    $APIMethod = "status?jobId=" + $Response

    while ((-Not($commandEndStatuses -contains $commandStatus)) -and
($maxIteration -lt 20))
    {
        Write-Host "Running try $( $maxIteration ) out of max 20 or until
successful"
        Write-Host "Call rest end point: $( $baseApiUrl )/$( $APIMethod )"
```

```
        $commandStatus = Invoke-WebRequest "$( $baseApiUrl )/$( $APIMethod )"
-Method GET -WebSession $session

        if ($commandStatus -like '*'+$commandSuccessStatus+'*')
        {
            Write-Host "Post process was successful"
            $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$success
            break
        }
        if ($commandStatus -like '*FAILED*')
        {
            Write-Host "Process status is failed"
            break
        }

        $finished = $commandEndStatuses -contains $commandStatus

        if (-Not$finished)
        {
            Write-Host "As the process status is not in a final state we will wait
5 seconds and try again"
            start-sleep -seconds 5
        }
        $maxIteration = $maxIteration + 1
    }

    if (-Not($commandStatus -like '*'+$commandSuccessStatus+'*')){
        Write-Host "Script execution failed"
        $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$failed
    }
}
catch [System.Net.WebException]
{
    Write-Host "Script execution failed"
Write-Host $($_.Exception)
    Write-Host "An exception was caught: $($_.Exception.Message)"
    $resultObj | Add-Member -MemberType NoteProperty -Name status -Value $failed
}

return $resultObj
```

6.16 Allocate reports

This sample script is used to allocate reports and includes another request that checks the allocation status. The request that checks the allocation status is posted every 5 seconds until the allocation process is finished, or more than 50 seconds have passed. When the script has finished, the allocation has been executed successfully.

```
#This script is used to allocate reports and includes another request which checks
the allocation status.
#The request that checks the allocation status is posted every 5 seconds until the
allocation is finished
#or more than 50 seconds have passed. When the script has finished, the allocation
has been executed successfully.

#specific input parameters for report allocation
param (
    [parameter(Position = 1, Mandatory = $true, ValueFromPipeline = $true,
```

```
ValueFromRemainingArguments = $false)]
    $preserveManual,
    [parameter(Position = 2, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $submissionId,
    [parameter(Position = 3, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $workspaceId,
    [parameter(Position = 4, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $securityToken,
    [parameter(Position = 5, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $reportingBaseUri
)

#create the cookie using the security token received from input parameters, you
should not change it
$session = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$cookie = New-Object System.Net.Cookie
$cookie.Name = "__Secure-WKFS-Token"
$cookie.Value = "$securityToken"
$failed = "FAILED"
$success = "SUCCESS"

$uriObject = [System.Uri]"$reportingBaseUri"
$cookie.Domain = $uriObject.Host
$cookie.Path = $uriObject.AbsolutePath

#add the cookie to the session object
$session.Cookies.Add($cookie);

################## Report allocation ##################

#'rh-api' is common for all available functionalities, you should not change it
$baseApiUrl = $reportingBaseUri + '/rh-api/external/v1_0/submissions/' +
$workspaceId +'/'+$submissionId
$APIMethod = "allocation?preserveManual="+$preserveManual

#call the rest endpoint for dataset creation and get the response
$resultObj = New-Object -TypeName psobject

$commandSuccessStatus = "COMPLETED"
$commandEndStatuses = $commandSuccessStatus, "STOPPED", "FAILED", "ABANDONED"
$commandStatus = "STARTING"
$maxIteration = 1
try
{
    Write-Host "Call the rest endpoint for allocation of report"
    $Response = Invoke-RestMethod  "$( $baseApiUrl )/$( $APIMethod )" -Method Post
-ContentType "application/json" -WebSession $session | ConvertTo-Json

    ################## Checking the status of allocation ##################

    $baseApiUrl = $reportingBaseUri + '/rh-api/external/v1_0/processes'

    $APIMethod = "status?jobId=" + $Response

    while ((-Not($commandEndStatuses -contains $commandStatus)) -and
($maxIteration -lt 20))
    {
        $commandStatus = Invoke-WebRequest "$( $baseApiUrl )/$( $APIMethod )"
```

```
-Method GET -WebSession $session
        if ($commandStatus -like '*'+$commandSuccessStatus+'*')
        {
            $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$success
            Write-Host "Allocation of reports successful"
            break
        }
        if ($commandStatus -like '*FAILED*')
        {
            Write-Host "Allocation of reports failed"
            break
        }
        $finished = $commandEndStatuses -contains $commandStatus

        if (-Not$finished)
        {
            start-sleep -seconds 5
             Write-Host "Wait 5 seconds and try again.."
        }
        $maxIteration = $maxIteration + 1
    }

    if (-Not($commandStatus -like '*'+$commandSuccessStatus+'*')){
        $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$failed
        Write-Host "Allocation of reports failed"
    }
}
catch [System.Net.WebException]
{
    Write-Host "Allocation of reports failed"
    Write-Host "An exception was caught: $($_.Exception.Message)"
    $resultObj | Add-Member -MemberType NoteProperty -Name status -Value $failed
}

return $resultObj
```

6.17 Validate reports

This sample script is used to validate reports and includes another request that checks the validation status. The request that checks the validation status is posted every 5 seconds until the validation process is finished, or more than 50 seconds have passed. When the script has finished, validation has been executed successfully.

```
#Script triggers a submission validation and checks if the validation is
successful or fails

#specific input parameters for report validation
param (
    [parameter(Position = 1, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $submissionId,
    [parameter(Position = 2, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $workspaceId,
    [parameter(Position = 3, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    $securityToken,
    [parameter(Position = 4, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
```

```
    $reportingBaseUri
)

#create the cookie using the security token received from input parameters, you
should not change it
$session = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$cookie = New-Object System.Net.Cookie
$cookie.Name = "__Secure-WKFS-Token"
$cookie.Value = "$securityToken"
$failed = "FAILED"
$success = "SUCCESS"

$uriObject = [System.Uri]"$reportingBaseUri"
$cookie.Domain = $uriObject.Host
$cookie.Path = $uriObject.AbsolutePath

#add the cookie to the session object
$session.Cookies.Add($cookie);

################## Report validation ##################

#create the api url for report validation
#'rh-api' is common for all available functionalities, you should not change it
$baseApiUrl = $reportingBaseUri + '/rh-api/external/v1_0/submissions/' +
$workspaceId + "/" + $submissionId + '/validation'

#call the rest endpoint for dataset creation and get the response
$resultObj = New-Object -TypeName psobject
try
{
    Write-Host "Call POST rest endpoint at: $( $baseApiUrl )"
    $jobId = Invoke-RestMethod  "$( $baseApiUrl )" -Method Post -ContentType
"application/json" -WebSession $session | ConvertTo-Json

    #################### Checking the status of validation
########################

    $commandSuccessStatus = "COMPLETED"
    $commandEndStatuses = $commandSuccessStatus, "NOT_VALIDATED", "INVALID"
    $commandStatus = "STARTING"

    $baseApiUrl = $reportingBaseUri + '/rh-api/external/v1_0'
    $APIMethod = "processes/status?jobId=" + $jobId
    Write-Host "Use the jobId $( $jobId ) response from the post call to check the
process status"
    $maxIteration = 1
    while ((-Not($commandEndStatuses -contains $commandStatus)) -and
($maxIteration -lt 20))
    {
        Write-Host "Running try $( $maxIteration ) out of max 20 or until
successful"
        Write-Host "Call rest endpoint $( $baseApiUrl )/$( $APIMethod ) to check
process status"
        $commandDetailsResponse = Invoke-WebRequest "$( $baseApiUrl )/$(
$APIMethod )" -Method GET -WebSession $session

        if ($commandDetailsResponse.StatusCode -eq 200)
        {
            $parsedCommandDetailsResponse = $commandDetailsResponse.Content

            $commandStatus = $parsedCommandDetailsResponse
            Write-Host "Process status is: "$commandStatus
```

```
            if ($commandStatus -eq $commandSuccessStatus)
            {
                break;
            }
        }
        else
        {
            break;
        }

        $finished = $commandEndStatuses -contains $commandStatus

        if (-Not$finished)
        {
            Write-Host "As the validation status is not in a final state we will
wait 5 seconds and try again"
            start-sleep -seconds 5
        }
        $maxIteration = $maxIteration + 1
    }

    if ($commandStatus -eq $commandSuccessStatus )
    {
        Write-Host "Validation completed successfully"
        $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$success
    }
    else
    {
        Write-Host "Script execution failed"
        $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$failed
    }
}
catch [System.Net.WebException]
{
    Write-Host "Script execution failed"
    Write-Host "An exception was caught: $($_.Exception.Message)"
    $resultObj | Add-Member -MemberType NoteProperty -Name status -Value $failed
}
return $resultObj
```

6.18 Create transmission file

This sample script is used to create transmission file based on submission and includes another request which checks the file creation status. The request that checks the creation status is posted every 5 seconds until transmission file creation is finished or more than 50 seconds have passed. When the script has finished, the file has been created successfully.

```
#Script triggers delivery process in RG TNG and checks if the process is
successful or fails

#specific input parameters for transmission
param (
    [parameter(Position = 1, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $submissionId,
    [parameter(Position = 2, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
```

```
    [ValidateNotNullOrEmpty()]
    $workspaceId,
    [parameter(Position = 3, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $securityToken,
    [parameter(Position = 4, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $reportingBaseUri
)

#create the cookie using the security token received from input parameters, you
should not change it
$session = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$cookie = New-Object System.Net.Cookie
$cookie.Name = "__Secure-WKFS-Token"
$cookie.Value = "$securityToken"

$uriObject = [System.Uri]"$reportingBaseUri"
$cookie.Domain = $uriObject.Host
$cookie.Path = $uriObject.AbsolutePath

#add the cookie to the session object
$session.Cookies.Add($cookie);

$resultObj = New-Object -TypeName psobject

################## Transmission ##################

#create the api url for dataset creation
#'rh-api' is common for all available functionalities, you should not change it
$baseApiUrl = $reportingBaseUri + '/rh-api/external/v1_0/submissions/'+
$workspaceId +'/'+ $submissionId +'/transmission'
$APIMethod = ""

$commandSuccessStatus = "COMPLETED"
$commandEndStatuses = $commandSuccessStatus, "STOPPED", "FAILED", "ABANDONED"
$commandStatus = "STARTING"
$maxIteration = 1
try
{
    Write-Host "Call rest endpoint $( $baseApiUrl )/$( $APIMethod ) to trigger the
creation of the transmission file"
    $Response = Invoke-RestMethod  "$( $baseApiUrl )/$( $APIMethod )" -Method Post
-ContentType "application/json" -WebSession $session | ConvertTo-Json

    ################## Checking the transmission status ##################

    $baseApiUrl = $reportingBaseUri + '/rh-api/external/v1_0/processes'

    $APIMethod = "status?jobId=" + $Response

    while ((-Not($commandEndStatuses -contains $commandStatus)) -and
($maxIteration -lt 20))
    {
        Write-Host "Running try $( $maxIteration ) out of max 20 or until
successful"
        Write-Host "Call rest end point: $( $baseApiUrl )/$( $APIMethod ) to
retrieve the transmission file creation process status"
        $commandStatus = Invoke-WebRequest "$( $baseApiUrl )/$( $APIMethod )"
-Method GET -WebSession $session
```

```
        if ($commandStatus -like '*'+$commandSuccessStatus+'*')
        {
            Write-Host "Process was successful"
            $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$success
            break
        }
        if ($commandStatus -like '*FAILED*')
        {
            break
        }

        $finished = $commandEndStatuses -contains $commandStatus

        if (-Not$finished)
        {
            Write-Host "As the process status is not in a final state we will wait
5 seconds and try again"
            start-sleep -seconds 5
        }
        $maxIteration = $maxIteration + 1
    }

    if (-Not($commandStatus -like '*'+$commandSuccessStatus+'*')){
        Write-Host "Process failed"
        $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$failed
    }
}
catch [System.Net.WebException]
{
    Write-Host "An exception was caught: $($_.Exception.Message)"
    $resultObj | Add-Member -MemberType NoteProperty -Name status -Value $failed
}

return $resultObj
```

6.19 Lookup and download the transmission files

This sample script is used to search for created transmission files. After the script has finished successfully, the names and paths of the transmission files will be returned. Next calls are made to download created transmission files.

```
#Script retrieves transmission artifacts.

#specific input parameters for download transmission files
param (
    [parameter(Position = 1, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $submissionId,
    [parameter(Position = 2, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $workspaceId,
    [parameter(Position = 3, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $securityToken,
    [parameter(Position = 4, Mandatory = $true, ValueFromPipeline = $true,
```

```
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $reportingBaseUri
)

#create the cookie using the security token received from input parameters, you
should not change it
$session = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$cookie = New-Object System.Net.Cookie
$cookie.Name = "__Secure-WKFS-Token"
$cookie.Value = "$securityToken"

$uriObject = [System.Uri]"$reportingBaseUri"
$cookie.Domain = $uriObject.Host
$cookie.Path = $uriObject.AbsolutePath

#add the cookie to the session object
$session.Cookies.Add($cookie);

$resultObj = New-Object -TypeName psobject
$failed = "FAILED"
$success = "SUCCESS"

################## Download transmission files ##################

#create the api url for download transmission file

$baseApiUrl = $reportingBaseUri + '/rh-api/external/v1_0/submissions/'+
$workspaceId +'/'+ $submissionId +'/transmission'

try
{
    Write-Host "Call rest endpoint $( $baseApiUrl ) to retrieve the transmission
artifacts path"
    $Response = Invoke-RestMethod  "$( $baseApiUrl )" -Method Get -ContentType
"application/json" -WebSession $session | ConvertTo-Json
    Write-Host "Response received: "$Response

    if ($Response)
    {
        $parsed = $Response | ConvertFrom-Json

        $noOfTransmissionFiles = $parsed.count
        $downloadableTransmissionFiles = 0

        foreach ($i in $parsed.value)
        {
            $transmissionFilePath = $i.path

            $baseApiUrl = $reportingBaseUri + '/rh-api/external/v1_0/' +
$transmissionFilePath
            Write-Host "Call rest endpoint $( $baseApiUrl ) to retrieve the
transmission artifact"
            $Response = Invoke-RestMethod  "$( $baseApiUrl )" -Method Get
-ContentType "application/json" -WebSession $session | ConvertTo-Json

            ################## Checking the status of download ##################
            if ($Response){
                Write-Host "Artifact retrieved"
                $downloadableTransmissionFiles = $downloadableTransmissionFiles +
1
            }
```

```
        }

        if ($noOfTransmissionFiles -eq $downloadableTransmissionFiles) {
            Write-Host "All artifacts were retrieved"
            $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$success
        } else {
            Write-Host "Download failed as not all files were retrieved"
            $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$failed
        }

    }
    else
    {
        exit -1
    }
}
catch [System.Net.WebException]
{
    Write-Host "An exception was caught: $($_.Exception.Message)"
    $resultObj | Add-Member -MemberType NoteProperty -Name status -Value $failed
}

return $resultObj
```

6.20 Prepare bridge structure

This sample script is used to prepare the data structure to launch the bridge.

```
#Script does REST call for preparation of data structure to launch bridge based on
the parameters provided
#specific input parameters for preparing the bridge structure
param (
    [parameter(Position = 1, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $submissionId,
    [parameter(Position = 2, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $workspaceId,
    [parameter(Position = 3, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $callerStringUniqueIdentifier,
    [parameter(Position = 4, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $securityToken,
    [parameter(Position = 5, Mandatory = $true, ValueFromPipeline = $true,
ValueFromRemainingArguments = $false)]
    [ValidateNotNullOrEmpty()]
    $reportingBaseUri
)

#create the cookie using the security token received from input parameters, you
should not change it
$session = New-Object Microsoft.PowerShell.Commands.WebRequestSession
$cookie = New-Object System.Net.Cookie
```

```
$cookie.Name = "__Secure-WKFS-Token"
$cookie.Value = "$securityToken"

$uriObject = [System.Uri]"$reportingBaseUri"
$cookie.Domain = $uriObject.Host
$cookie.Path = $uriObject.AbsolutePath

#add the cookie to the session object
$session.Cookies.Add($cookie);
$resultObj = New-Object -TypeName psobject

################## Prepare Bridge Structure ##################

$baseApiUrl = $reportingBaseUri + '/rh-api/external/v1_0/submissions/' +
$workspaceId + '/' + $submissionId + '/databridge/' +
$callerStringUniqueIdentifier
try
{
    Write-Host "Call rest endpoint $( $baseApiUrl )"
    $Response = Invoke-RestMethod  "$( $baseApiUrl )" -Method Put -ContentType
"application/json" -WebSession $session | ConvertTo-Json
    Write-Host "Response received: "$Response
    if ($Response)
    {
       Write-Host "Rest call response was successful"
       $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$success
    }
    else
    {
        Write-Host "Rest call response failed"
        $resultObj | Add-Member -MemberType NoteProperty -Name status -Value
$failed
    }
}
catch [System.Net.WebException]
{
    Write-Host "An exception was caught: $($_.Exception.Message)"
    $resultObj | Add-Member -MemberType NoteProperty -Name status -Value $failed
}
return $resultObj
```

6.21 Skip SSL validation

Use the following script to skip a server SSL validation.

```
########################## Certificate SSL Error handler  #######################
$code = @"
public class SSLHandler
{
    public static System.Net.Security.RemoteCertificateValidationCallback
GetSSLHandler()
    {
        return new
System.Net.Security.RemoteCertificateValidationCallback((sender, certificate,
chain, policyErrors) => { return true; });
    }
}
"@
#compile the class
```

```
Add-Type -TypeDefinition $code

$AllProtocols = [System.Net.SecurityProtocolType]'Ssl3,Tls,Tls11,Tls12'
[System.Net.ServicePointManager]::SecurityProtocol = $AllProtocols
[System.Net.ServicePointManager]::ServerCertificateValidationCallback =
[SSLHandler]::GetSSLHandler()
```